PROJECT REPORT

CareerAI - AI-Powered Career Guidance and Resume-Based Interview System

Submitted in partial fulfillment of the requirements for the Diploma in Computer Engineering

Institute:
[INSTITUTE NAME]

Department:
Department of Computer Engineering

Academic Year:
2026-27

Submitted By:
1. [STUDENT NAME]
2. [STUDENT NAME]
3. [STUDENT NAME]
4. [STUDENT NAME]
5. [STUDENT NAME]

Guided By:
[GUIDE NAME]

Certificate

This is to certify that the project report entitled CareerAI - AI-Powered Career Guidance and Resume-Based Interview System is a bonafide work carried out by the students listed on the cover page in partial fulfillment of the requirements for the Diploma in Computer Engineering during the academic year 2026-27. The work has been prepared for academic submission and viva evaluation.

Place: [PLACE]
Date: [DATE]

Guide: [GUIDE NAME]
Head of Department: [HOD NAME]
External Examiner: [EXAMINER NAME]

Declaration

We hereby declare that the project report entitled CareerAI - AI-Powered Career Guidance and Resume-Based Interview System has been prepared by us under the guidance of [GUIDE NAME]. The report is based on the actual Django project implementation submitted with this work. All sources of technical reference have been acknowledged in the References section. The personal academic details not available in the source archive have been kept as editable placeholders.

Acknowledgement

We express our sincere gratitude to our project guide [GUIDE NAME] for valuable guidance, encouragement, and technical support throughout the development of this project. We also thank the Department of Computer Engineering and [INSTITUTE NAME] for providing the academic environment required to complete this work. We are thankful to our classmates, friends, and family members for their cooperation during analysis, implementation, and testing.

Abstract

CareerAI is a Django-based web application developed to support students and job seekers in career planning, resume analysis, and interview preparation. The system allows a registered user to upload a PDF resume, extracts readable text using pypdf, builds a local profile of skills, education, experience, projects, and certifications, and then uses that evidence to generate career suggestions and resume-grounded interview questions. The implementation includes user registration, login, settings, PDF-only resume upload validation, password recovery through a short-lived OTP, performance analytics, interview scoring, feedback generation, bilingual professional answer generation, and PDF interview report download. The default implementation is rule-based and local, while optional provider hooks for Gemini, OpenAI-compatible, or OpenRouter APIs are present only when explicitly enabled through environment variables. The project uses Django 6, SQLite, HTML, CSS, JavaScript, pypdf, ReportLab, and Django authentication features. This report documents the actual modules, models, workflows, security mechanisms, testing evidence, limitations, and future scope of the submitted source code.

Table of Contents

- Chapter 1 Introduction
- Chapter 2 Literature and Existing System Study
- Chapter 3 Requirement Analysis
- Chapter 4 System Analysis and Design
- Chapter 5 Technology Stack
- Chapter 6 System Modules
- Chapter 7 Database Design
- Chapter 8 Implementation
- Chapter 9 Algorithms and Logic
- Chapter 10 User Interface and Screenshots
- Chapter 11 Testing
- Chapter 12 Security
- Chapter 13 Results and Discussion
- Chapter 14 Advantages Limitations and Future Scope
- Chapter 15 Conclusion
- References
- Appendix

List of Figures

- Figure 4.1 System Architecture
- Figure 4.2 Overall Workflow
- Figure 4.3 Context Diagram
- Figure 4.4 DFD Level 0
- Figure 4.5 DFD Level 1
- Figure 4.6 Use Case Diagram
- Figure 4.7 Activity Diagram
- Figure 4.8 Sequence Diagram
- Figure 4.9 Component Diagram
- Figure 4.10 ER Diagram
- Figure 9.1 Resume Processing Flowchart
- Figure 9.2 Interview Evaluation Flowchart
- Figure 10.1 to 10.12 Screenshot Placeholders

List of Tables

- Table 2.1 Existing System vs Proposed System
- Table 3.1 Functional Requirements
- Table 3.2 Non Functional Requirements
- Table 5.1 Technology Stack

- Table 7.1 to 7.7 Data Dictionary
- Table 8.1 URL Routing
- Table 11.1 Automated Test Result Summary
- Table 11.2 Proposed Manual Test Cases

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence, used in the project name and optional provider terminology
API	Application Programming Interface
CSRF	Cross Site Request Forgery
DFD	Data Flow Diagram
DRF	Django REST Framework
ER	Entity Relationship
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
OTP	One Time Password
PDF	Portable Document Format
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Introduction

CareerAI is a student project that addresses a practical academic and career-readiness problem. Diploma students often need guidance about suitable job roles, portfolio gaps, and interview preparation, but their preparation is frequently generic and not connected to the content of their own resume. The submitted system provides a web-based workflow where the user's PDF resume becomes the evidence source for career suggestions and practice interview questions.

1.2 Background

Career guidance and interview preparation are normally handled through teachers, placement officers, mentors, online articles, and mock interviews. These methods are useful, but students may still find it difficult to convert their own skills and project work into clear interview answers. The project therefore combines a resume upload feature, deterministic text analysis, question generation, answer scoring, feedback, and a downloadable interview report.

1.3 Problem Statement

Students require a simple platform that can read a resume, identify visible skills and project evidence, suggest career improvement areas, and provide interview practice linked to that evidence. The system should not replace a professional counsellor; it should help a learner practise and organize preparation in a structured way.

1.4 Motivation

The motivation behind CareerAI is to make interview practice more personal than a static question bank. A resume-based interview is useful because it asks the candidate to defend real claims already written in the resume.

1.5 Need for the System

The need for the system comes from gaps in ordinary preparation: resumes are often not reviewed technically, interview questions are often generic, and students may not know how to explain projects using situation, action, result, and learning.

1.6 Objectives

The implemented objectives are user registration and authentication, resume upload, local PDF text extraction, profile construction from resume text, career guidance, resume-grounded interview generation, answer evaluation, feedback, performance history, password reset with OTP, and PDF report generation.

1.7 Scope of the Project

The scope is a Django web application for local or college-level deployment. It supports individual authenticated users and stores their resumes, interview sessions, answers, feedback, and settings in SQLite. It is suitable for academic demonstration and controlled use.

1.8 Proposed Solution

The proposed solution is a modular Django application named CareerAI. The users app handles registration, login, settings, resume upload, career guidance, interview APIs, report pages, and PDF report download. Local logic in resume_services.py performs the core resume analysis and interview processing.

1.9 Advantages

The system is simple to deploy, works in local fallback mode without paid APIs, grounds interview questions in resume evidence, provides bilingual model answers, and stores performance history per user.

1.10 Limitations

The default logic is rule-based and depends on selectable PDF text. It does not perform OCR, does not guarantee professional counselling quality, does not include a production-grade rate limiter, and sends resume text to an external provider only if optional AI mode is enabled by configuration.

1.11 Future Scope

Future work may include OCR, stronger resume parsing, rate limiting, malware scanning for uploads, richer analytics, mobile application support, cloud deployment, and job recommendation integration.

1.12 Organization of the Report

The remaining chapters discuss existing systems, requirements, analysis and design, technologies, modules, database design, implementation details, algorithms, interface screens, testing, security, results, future scope, conclusion, references, and appendices.

CHAPTER 2
LITERATURE AND EXISTING SYSTEM STUDY

2.1 Existing Career Guidance Methods

Existing career guidance usually includes faculty counselling, placement-cell sessions, aptitude tests, online articles, and peer advice. These methods can guide broad choices, but they may not directly analyze the student's own resume evidence.

2.2 Existing Resume Screening and Analysis Approaches

Resume analysis systems usually parse documents, detect skills, and compare them with role requirements. In this project, the implemented parser is conservative and local: it extracts PDF text using pypdf and detects known skills and sections through string and regular-expression logic.

2.3 Existing Interview Preparation Platforms

Common interview platforms provide question banks, mock interviews, answer samples, and sometimes voice-based practice. CareerAI implements a resume-grounded interview flow rather than only static questions.

2.4 Problems in Existing Approaches

Existing approaches may give the same questions to all students, require paid services, or fail to connect answers with resume evidence. Some tools also overclaim AI capability without explaining the logic used.

2.5 Existing System vs Proposed System

The comparison below presents the actual proposed implementation without inventing market statistics.

Aspect	Traditional / Existing Approach	CareerAI Implementation
Resume basis	Often manual or generic	Reads uploaded PDF and stores raw_text and parsed_profile
Question source	Static question bank	Local resume-grounded generation with optional provider fallback
Scoring	Manual judgement	Rule-based technical, relevance, communication and filler scoring
Report	May be verbal only	HTML report and downloadable ReportLab PDF
Cost	May depend on paid services	Default local mode works without API keys

2.6 Research and Technology Gap

The main gap addressed by this project is the connection between resume evidence and interview preparation. The implementation avoids unsupported claims and uses deterministic processing as the baseline.

2.7 Summary

CareerAI is positioned as a practical student support system rather than a replacement for counsellors, recruiters, or professional interviewers.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Functional Requirements

Requirement	Implemented	Evidence
User registration with email and password	Yes	RegisterForm and register view
User login and logout	Yes	user_login and user_logout views
Password reset using OTP	Yes	forgot_password, verify_password_otp, resend_password_otp, password_reset_new
User settings	Yes	UserSettings model and settings_page view
PDF resume upload	Yes	upload_resume view with extension, size, and PDF header checks
PDF text extraction	Yes	extract_resume_text using pypdf
Resume profile analysis	Yes	build_local_profile
Career guidance	Yes	career_guidance_page and generate_career_guidance
Resume-based interview	Yes	interview_start_api, interview_answer_api, interview_finish_api
Interview PDF report	Yes	build_interview_report_pdf with ReportLab
OCR for scanned resumes	No	Not present in source code
Job portal integration	No	Future enhancement only

3.2 Non Functional Requirements

Requirement	Implementation Status
Security	Django auth, CSRF middleware, password hashing, OTP hashing, upload validation, secure cookie settings
Usability	HTML templates, CSS pages, dashboard navigation, settings, performance page
Maintainability	Separate files for models, forms, views, resume services, PDF reports

Portability	SQLite and environment-variable settings support local deployment
Availability	No explicit production availability mechanism implemented

3.3 Hardware Requirements

Minimum recommended hardware for local demonstration is a standard computer with a modern browser, 4 GB RAM, and enough disk space for Python packages, SQLite database, static files, and uploaded resumes.

3.4 Software Requirements

The source uses Python, Django 6, Django REST Framework, pypdf, ReportLab, python-dotenv, SQLite, HTML, CSS, and JavaScript. A virtual environment is recommended.

3.5 User Requirements

A user should be able to create an account, sign in, upload a readable PDF resume, start an interview, answer questions, view scores and feedback, download a report, update settings, and reset a forgotten password.

3.6 System Constraints

The resume must be a valid PDF with selectable text. The default mode is rule-based. External AI providers are optional and require explicit environment configuration.

3.7 Feasibility Study

Technical feasibility is high for academic deployment because Django and SQLite are sufficient. Economic feasibility is high because the local mode avoids paid API services. Operational feasibility is acceptable because the workflow is browser based. Schedule feasibility is reasonable because the project is organized as a single Django app with focused modules.

CHAPTER 4
SYSTEM ANALYSIS AND DESIGN

4.1 System Architecture

Figure 4.1 System Architecture

```
Browser UI
  |
  | HTTP / JSON / CSRF
  v
Django URL Router
  |
  +-- Template Views: home, dashboard, settings, upload, guidance, reports
  +-- Interview APIs: start, answer, finish, history
  |
  v
users app services
  +-- resume_services.py: extraction, profile, questions, scoring, feedback
  +-- pdf_reports.py: ReportLab interview PDF
  |
  v
SQLite database + media/resumes
```

4.2 Overall System Workflow

Figure 4.2 Overall System Workflow

```
Register/Login -> Update settings/profile -> Upload PDF resume -> Validate file -> Extract text -> Build parsed
profile -> Generate guidance or start interview -> Save questions -> Submit answers -> Calculate scores ->
Finish session -> Save feedback and model answers -> View HTML report or PDF report
```

4.3 Context Diagram

Figure 4.3 Context Diagram

```
Student/User <-> CareerAI Web Application <-> SQLite Database
CareerAI Web Application -> Email Backend for OTP
CareerAI Web Application -> Optional AI Provider only when ENABLE_AI=true and provider settings are configured
```

4.4 Data Flow Diagram Level 0

Figure 4.4 Data Flow Diagram Level 0

```
User Input -> Authentication Process -> User Account Data
Resume File -> Resume Processing -> Resume and Parsed Profile Data
Interview Request -> Question Generation -> Interview Session Data
Answer Text -> Answer Evaluation -> Scores and Feedback
Completed Session -> Report Generation -> HTML/PDF Output
```

4.5 Data Flow Diagram Level 1

Figure 4.5 Data Flow Diagram Level 1

```
Upload Resume: Request.FILES -> extension check -> size check -> PDF signature check -> Resume row -> pypdf
extraction -> raw_text -> build_local_profile -> parsed_profile
Interview: JSON start request -> latest resume -> generate_questions -> InterviewSession + InterviewQuestion
rows -> answer JSON -> evaluate_answer -> InterviewAnswer -> finish -> InterviewFeedback
```

4.6 Use Case Diagram

Figure 4.6 Use Case Diagram

```
Actor: Registered User
Use cases: Register, Login, Reset Password, Update Settings, Upload Resume, Generate Career Guidance, Start
Interview, Submit Answer, Finish Interview, View Performance, View Report, Download PDF, Logout
Actor: Admin
Use cases: Access Django admin for model administration
```

4.7 Activity Diagram

Figure 4.7 Activity Diagram

```
Start -> User logs in -> Upload resume? -> If no, request PDF upload -> Validate PDF -> Extract text -> Build
profile -> User selects career guidance or interview -> If interview, generate questions -> submit answers ->
score answers -> finish -> show report -> End
```

4.8 Sequence Diagram

Figure 4.8 Sequence Diagram

```
User -> Browser: click Start Interview
Browser -> Django API: POST /api/interview/start/
Django -> Resume model: fetch latest resume
Django -> resume_services: generate_questions
Django -> Database: create session and questions
Django -> Browser: return questions JSON
User -> Browser: submits answer
Browser -> Django API: POST /api/interview/answer/
Django -> resume_services: evaluate_answer
Django -> Database: save InterviewAnswer
```

4.9 Component Diagram

Figure 4.9 Component Diagram

```
career_guidance project
  manage.py
  career_guidance/settings.py and urls.py
  users/models.py
  users/forms.py
  users/views.py
  users/resume_services.py
  users/pdf_reports.py
  users/templates/users/*.html
  users/static/users/*.css and landing.js
  users/migrations/*.py
```

4.10 ER Diagram

Figure 4.10 ER Diagram

```
auth.User 1--1 Student
auth.User 1--1 UserSettings
auth.User 1--many Resume
auth.User 1--many InterviewSession
Resume 1--many InterviewSession
InterviewSession 1--many InterviewQuestion
InterviewQuestion 1--1 InterviewAnswer
InterviewSession 1--1 InterviewFeedback
```

4.11 Database Design

The project uses Django ORM models backed by SQLite. Authentication uses Django's built-in auth_user table. The application-specific models are Student, UserSettings, Resume, InterviewSession, InterviewQuestion, InterviewAnswer, and InterviewFeedback.

4.12 Data Dictionary

A detailed data dictionary is provided in Chapter 7.

CHAPTER 5
TECHNOLOGY STACK

Technology	What it is	Use in CareerAI
Python	General purpose programming language	Back-end application logic and services
Django 6	Python web framework	Routing, views, ORM, templates, authentication, sessions, security middleware
Django REST Framework	API support package	Installed as a third-party app; interview APIs are implemented with Django JsonResponse
SQLite	File-based relational database	Default database configured as BASE_DIR / db.sqlite3
HTML	Markup language	Templates for landing, login, dashboard, upload, guidance, interview, reports
CSS	Style sheet language	Separate static CSS files for major pages
JavaScript	Browser scripting language	Landing animation and interview UI API calls, speech synthesis/recognition support in template
pypdf	PDF reading library	Extracts selectable text from uploaded PDF resumes
ReportLab	PDF generation library	Creates downloadable interview reports
python-dotenv	Environment variable helper	Loads .env files when available
Email backend	Django email system	Sends password reset OTP through SMTP or prints to console backend

Optional provider settings exist for Gemini, OpenAI-compatible APIs, and OpenRouter, but the default project configuration keeps AI disabled and provider set to local. Therefore, the report treats external AI as optional configuration, not as a guaranteed implemented runtime feature.

CHAPTER 6
SYSTEM MODULES

6.1 Landing and Home Module

Public landing page served by landing_page and index.html. It introduces resume intelligence, adaptive questions, voice interview room, bilingual coaching, scoring, and report download as product concepts shown in the UI.

6.2 User Registration

RegisterForm collects username, email, and password. Email is required and must be unique. The view validates password strength using Django validators and stores the password through set_password.

6.3 User Login and Logout

user_login authenticates username and password and redirects valid users to dashboard. Invalid or malformed requests return the login page with an error message. user_logout signs out and redirects to login.

6.4 Password Recovery and OTP

forgot_password sends a six-digit OTP, stores only a hashed OTP in session, applies expiry, limits attempts, supports resend cooldown, and invalidates existing sessions after successful reset.

6.5 User Profile and Settings

settings_page creates or updates Student and UserSettings records. It stores full name, college, course, email, notification options, theme, and profile visibility.

6.6 Resume Upload

upload_resume accepts a file named resume, requires .pdf extension, checks size using RESUME_MAX_SIZE, verifies the PDF signature starts with %PDF-, saves Resume, extracts text, and stores parsed profile.

6.7 Resume Text Extraction

extract_resume_text uses pypdf.PdfReader to extract selectable text from every page. Empty extracted text raises ResumeProcessingError.

6.8 Resume Profile Analysis

build_local_profile detects skills from SKILL_CATALOG and extracts education, experience, projects, and certifications from recognized resume sections.

6.9 Career Guidance

career_guidance_page uses the latest resume and optional latest interview feedback to produce target roles, strengths, gaps, project ideas, a thirty-day plan, and next steps.

6.10 Resume-Based Interview

interview_start_api creates a session and stores generated questions. Easy and medium interviews are clamped to 15 to 30 questions; hard interviews use 40 questions.

6.11 Answer Evaluation

interview_answer_api evaluates answer text using expected topics, word count, communication estimate, and filler words. It saves technical, relevance, communication, and overall scores.

6.12 Interview Feedback

interview_finish_api averages submitted answer scores, generates strengths, weaknesses, recommendations, and professional bilingual answers, then marks the session completed.

6.13 Interview Report

interview_report renders a protected HTML report for completed sessions owned by the signed-in user.

6.14 PDF Report Generation

interview_report_pdf calls build_interview_report_pdf, which uses ReportLab to generate a styled report with score summary, feedback, questions, model answers, and candidate answers.

6.15 Performance Module

performance_page calculates completed interview count, average score, best score, latest score, improvement delta, category scores, trend data, and recent session links for the current user only.

CHAPTER 7
DATABASE DESIGN

The database is SQLite as configured in settings.py. Django also creates standard authentication and session tables. The application models are documented below.

7.x Student Model

Field	Type	Purpose
id	BigAutoField	Primary key
user	OneToOneField User	Student belongs to one user
full_name	CharField(100)	Student name
college	CharField(100)	College name
course	CharField(100, blank)	Course name
cgpa	FloatField	Academic score
skills	TextField(blank)	Manually entered skills

7.x UserSettings Model

Field	Type	Purpose
id	BigAutoField	Primary key
user	OneToOneField User	Settings owner
career_notifications	BooleanField	Career notification preference
resume_notifications	BooleanField	Resume notification preference
job_notifications	BooleanField	Job notification preference
theme	CharField choices	light, dark, or system
profile_visibility	BooleanField	Profile visibility preference

7.x Resume Model

Field	Type	Purpose
id	BigAutoField	Primary key
user	ForeignKey User	Owner

resume_file	FileField	Uploaded under resumes/
raw_text	TextField	Extracted PDF text
parsed_profile	JSONField	Detected skills, education, projects, experience
parse_status	CharField choices	pending, complete, failed
parse_error	CharField(255)	Extraction error
uploaded_at	DateTimeField	Upload timestamp

7.x InterviewSession Model

Field	Type	Purpose
id	BigAutoField	Primary key
user	ForeignKey User	Owner
resume	ForeignKey Resume nullable	Resume evidence
role	CharField(100)	Target role
difficulty	CharField(20)	easy, medium, hard
interview_type	CharField(30)	general, technical, behavioral
status	CharField(20)	in_progress or completed
question_count	PositiveSmallIntegerField	Number of questions
overall_score	FloatField nullable	Final score
started_at	DateTimeField	Start time
completed_at	DateTimeField nullable	Finish time

7.x InterviewQuestion Model

Field	Type	Purpose
id	BigAutoField	Primary key
session	ForeignKey InterviewSession	Parent session
order	PositiveSmallIntegerField	Question order
question_text	TextField	Question
category	CharField(50)	Question category
source	CharField(30)	local, ai, project, skill, fallback

source_evidence	CharField(255)	Resume evidence phrase
expected_topics	JSONField	Expected answer topics

7.x InterviewAnswer Model

Field	Type	Purpose
id	BigAutoField	Primary key
question	OneToOneField InterviewQuestion	One answer per question
answer_text	TextField	Candidate answer
duration_seconds	PositiveIntegerField nullable	Optional duration
technical_score	FloatField	Technical score
relevance_score	FloatField	Relevance score
communication_score	FloatField	Communication score
overall_score	FloatField	Overall score
filler_count	PositiveIntegerField	Detected filler words
feedback	JSONField	Feedback items
submitted_at	DateTimeField	Last submission time

7.x InterviewFeedback Model

Field	Type	Purpose
id	BigAutoField	Primary key
session	OneToOneField InterviewSession	Completed session
strengths	JSONField	Strength list
weaknesses	JSONField	Weakness list
recommendations	JSONField	Recommendations
professional_answers	JSONField	Bilingual model answers
generated_by	CharField(20)	local or provider indicator
created_at	DateTimeField	Generation time

7.10 ER Diagram

Figure 7.1 ER Diagram

```
auth.User 1--1 Student
auth.User 1--1 UserSettings
auth.User 1--many Resume
auth.User 1--many InterviewSession
Resume 1--many InterviewSession
InterviewSession 1--many InterviewQuestion
InterviewQuestion 1--1 InterviewAnswer
InterviewSession 1--1 InterviewFeedback
```

CHAPTER 8
IMPLEMENTATION

Route	View	Purpose
/	landing_page	Public landing page
/register/	register	Create account
/login/	user_login	Authenticate user
/dashboard/	dashboard	Main user dashboard
/upload/	upload_resume	Upload and parse PDF resume
/career-guidance/	career_guidance_page	Generate career suggestions
/ai-interview/	ai_interview_page	Interview interface
/api/interview/start/	interview_start_api	Create questions
/api/interview/answer/	interview_answer_api	Score answer
/api/interview/finish/	interview_finish_api	Complete session
/interview/<id>/report/	interview_report	HTML report
/interview/<id>/report.pdf	interview_report_pdf	PDF download

Resume Upload Validation

```
extension = os.path.splitext(resume.name or '')[1].lower()
if extension != '.pdf':
    return render(..., {'error': 'Only PDF resumes are supported.'})
header = resume.read(5)
resume.seek(0)
if header != b'%PDF-':
    return render(..., {'error': 'The uploaded file is not a valid PDF.'})
```

PDF Text Extraction

```
from pypdf import PdfReader
with file_field.open('rb') as handle:
    reader = PdfReader(handle)
    pages = [(page.extract_text() or '').strip() for page in reader.pages]
text = '\n'.join(page for page in pages if page).strip()
```

Answer Scoring Formula

```
technical = round((topic_score * 0.7) + (length_score * 0.3), 2)
relevance = round(topic_score, 2)
overall = round((technical * 0.5) + (relevance * 0.25) + (communication * 0.25), 2)
```

PDF Report Response

```
pdf_bytes = build_interview_report_pdf(session)
response = HttpResponse(pdf_bytes, content_type='application/pdf')
response['Content-Disposition'] = f'attachment; filename="careerai_interview_report_{session.id}.pdf"'
```

Templates are stored under users/templates/users and static styling is stored under users/static/users. The interview template includes JavaScript fetch calls to the JSON endpoints and browser speech features. Error handling is implemented through form validation, JSON status codes, Django messages, and ResumeProcessingError.

CHAPTER 9
ALGORITHMS AND LOGIC

9.1 Resume Text Extraction Algorithm

Input: uploaded PDF FileField. Processing: open file, read all pages using pypdf, extract selectable text, join non-empty page strings, reject empty text, limit stored text to 100000 characters. Output: raw resume text or ResumeProcessingError.

9.2 Skill Detection Algorithm

Input: raw resume text and optional Student.skills. Processing: compare text against SKILL_CATALOG using token-aware regular expression matching, merge manually entered skills, remove duplicates case-insensitively. Output: list of detected skills.

9.3 Section Detection Algorithm

Input: raw text lines. Processing: normalize each line, detect headings matching skills, education, experience, projects, or certifications aliases, collect following lines under the active section. Output: dictionary of section lines.

9.4 Interview Question Generation Algorithm

Input: parsed profile, role, difficulty, interview type, count, language. Processing: build project, skill, education, experience, motivation, and generic profile questions from evidence; fill sparse resumes with variant templates; optionally call configured provider and validate evidence before accepting. Output: ordered question list.

9.5 Answer Evaluation Algorithm

Input: question expected topics and answer text. Processing: tokenize answer, count filler words, match expected topics, compute topic score, length score, communication score, technical score, relevance score, and weighted overall score. Output: numeric scores and feedback list.

9.6 Feedback Generation Algorithm

Input: saved answer rows. Processing: mark strengths for high scores, weaknesses for low score or filler count, and recommendations based on detected weaknesses. Output: strengths, weaknesses, and recommendations.

9.7 OTP Reset Algorithm

Input: registered email. Processing: lookup active unique user without disclosing existence, generate six-digit OTP, store password hash of OTP in session, email OTP, verify attempts and expiry, validate new password, save new hash, invalidate old sessions. Output: reset success or safe error message.

Figure 9.1 Resume Processing Flowchart

```
Start
  -> Receive PDF
  -> Validate extension, size, signature
  -> Extract text using pypdf
  -> Detect sections and skills
  -> Store raw_text and parsed_profile
End
```

Figure 9.2 Interview Evaluation Flowchart

```
Start
  -> Receive answer JSON
  -> Load owned question
  -> Count words and filler terms
  -> Match expected topics
  -> Calculate technical, relevance, communication, overall
  -> Save InterviewAnswer
End
```

CHAPTER 10
USER INTERFACE AND SCREENSHOTS

This chapter presents screenshots captured from the actual CareerAI Django application running locally on the development server. A demo account and demo resume were used so that no personal password, private resume, or API key is exposed.

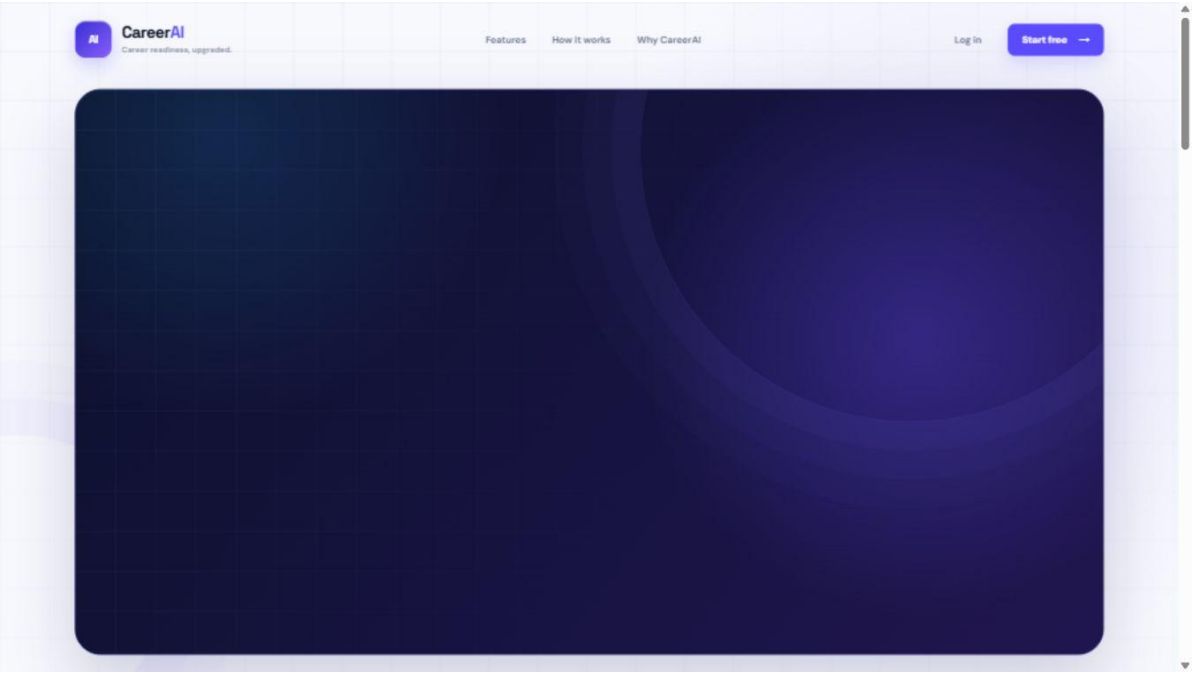


Figure 10.1: CareerAI Home Page

The home page is the public entry point served by the landing_page view and index.html template. It introduces the main workflow and provides navigation links to registration and login.

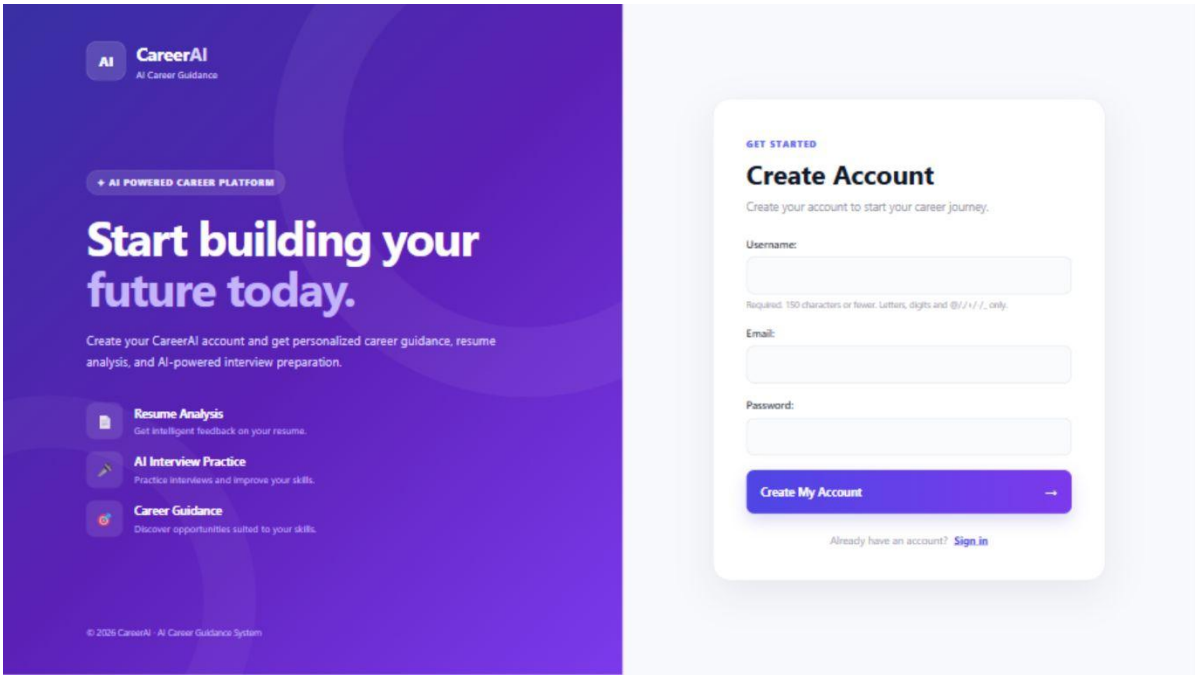


Figure 10.2: User Registration Interface

The registration screen belongs to the RegisterForm module. The user provides username, email address, and password before the system validates and creates a Django auth user.

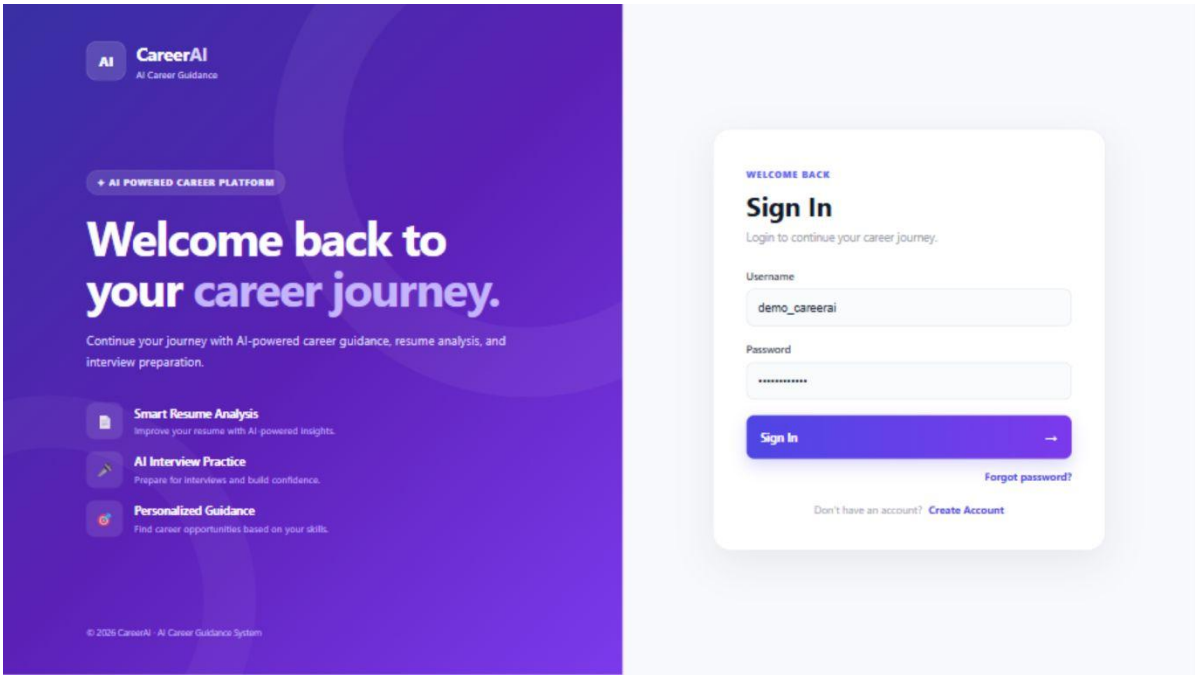


Figure 10.3: User Login Interface

The login page uses the user_login view. It accepts username and password and creates an authenticated Django session for valid credentials.

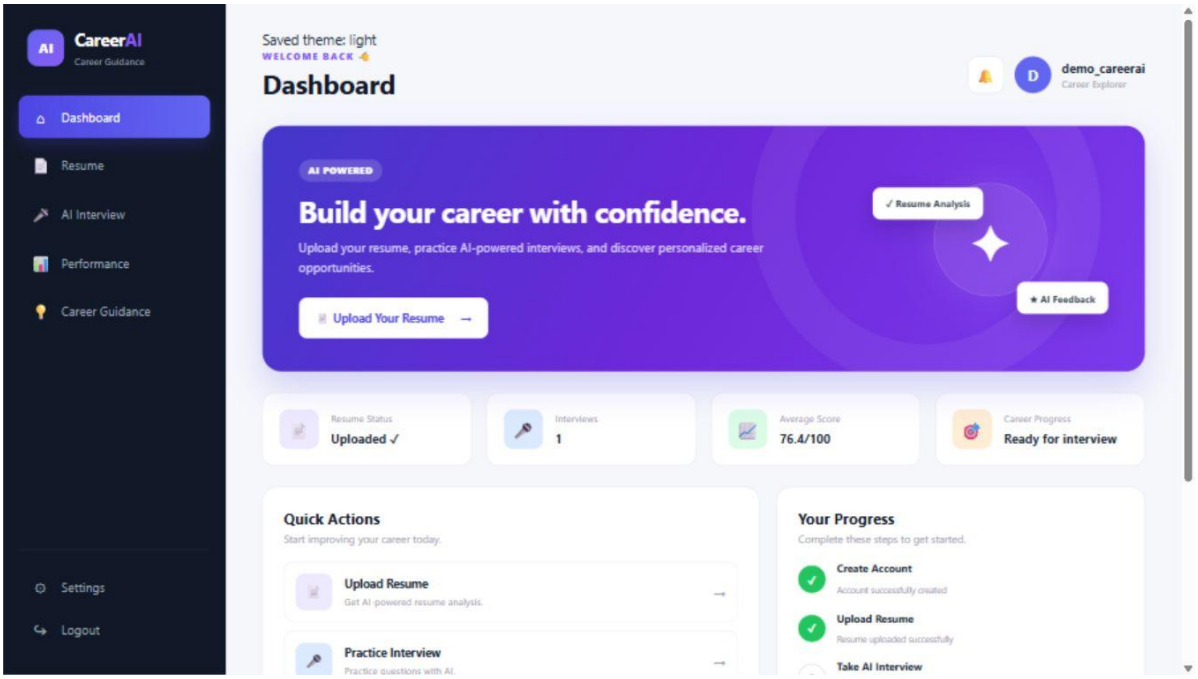


Figure 10.4: Authenticated Dashboard

The dashboard is the main user workspace. It shows resume status, interview count, average score information, and links to upload, interview, performance, career guidance, and settings modules.

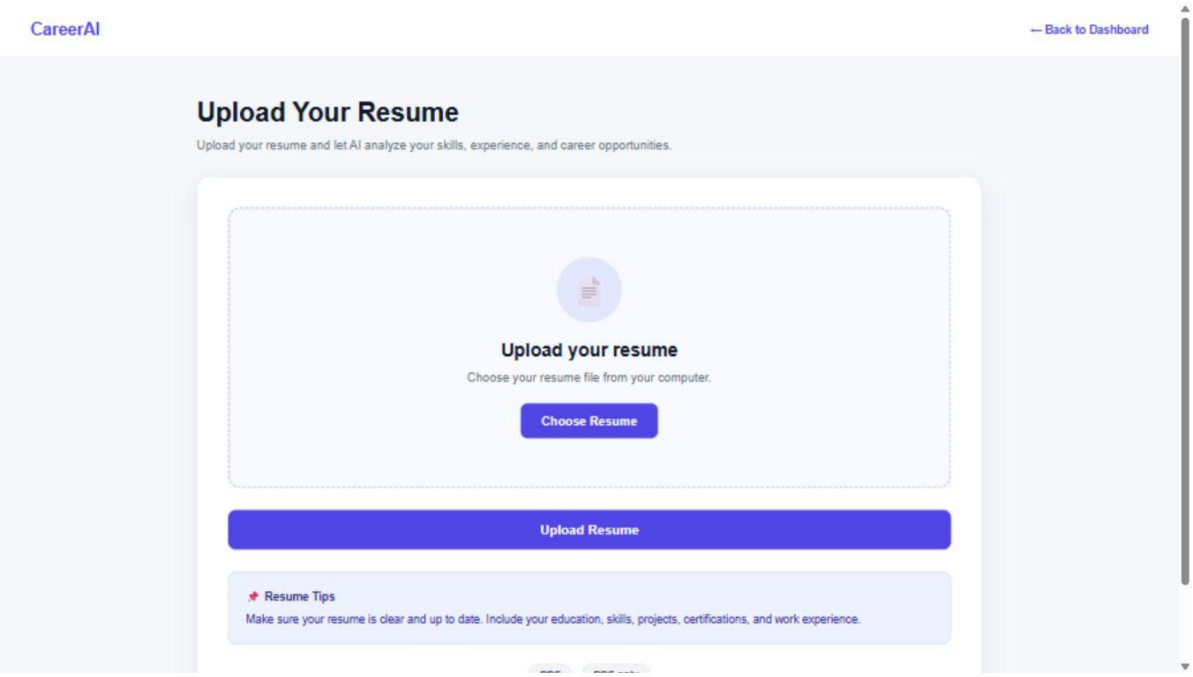


Figure 10.5: Resume Upload Interface

The resume upload screen accepts a PDF file and is connected with server-side validation for extension, file size, and PDF signature. After upload, the system extracts text and builds a parsed profile.

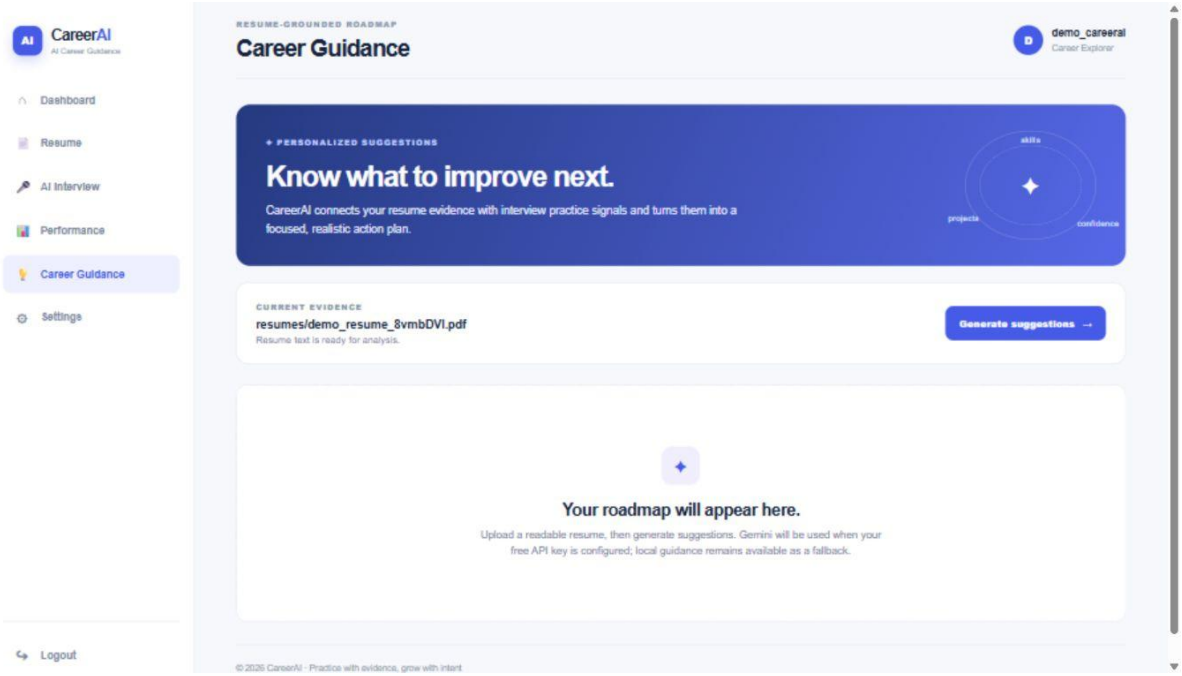


Figure 10.6: Career Guidance Request Screen

This screen shows the career guidance page before generating suggestions. It displays the current resume evidence and offers the action to generate guidance.

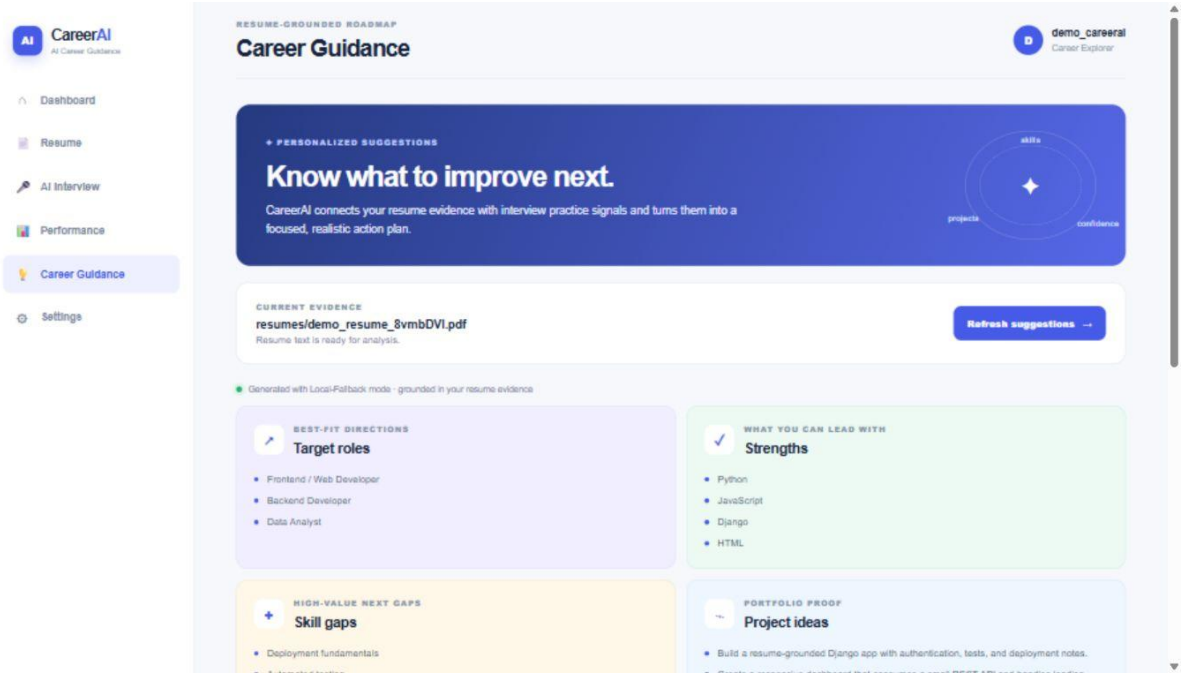


Figure 10.7: Career Recommendations Screen

The recommendations view shows generated target roles, strengths, gaps, project ideas, and a thirty-day plan based on the latest parsed resume and optional interview feedback.

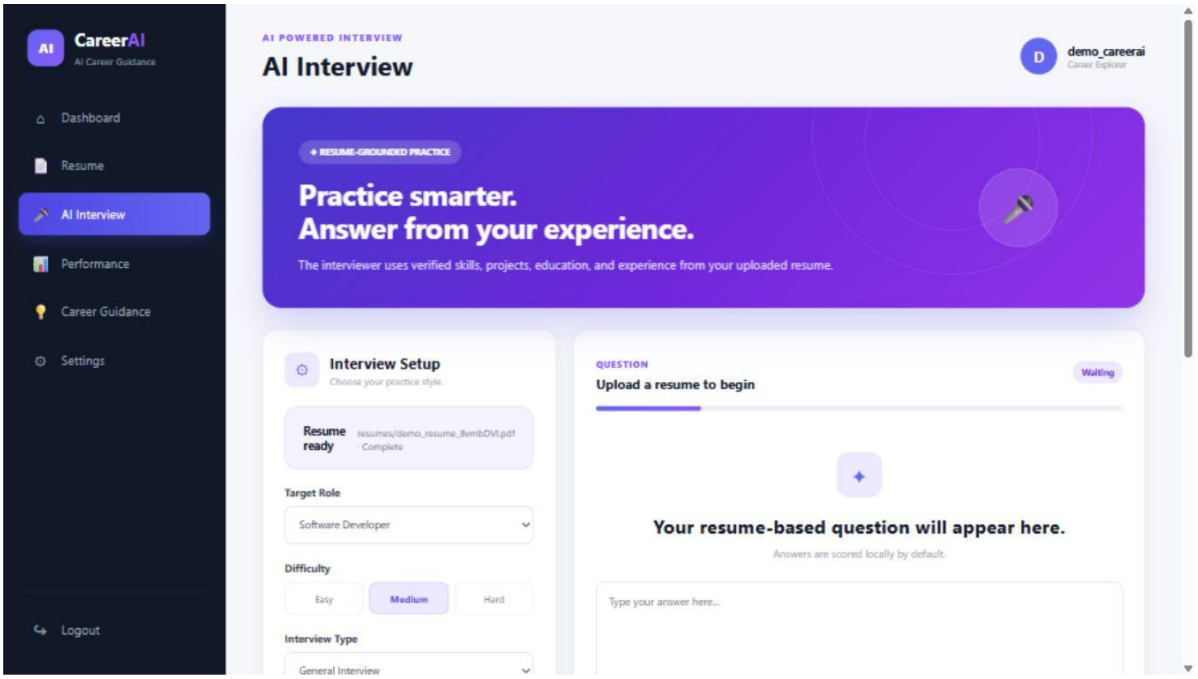


Figure 10.8: AI Interview Setup Screen

The interview page allows the user to select role, difficulty, interview type, language, and question count. It calls the interview JSON APIs through JavaScript.

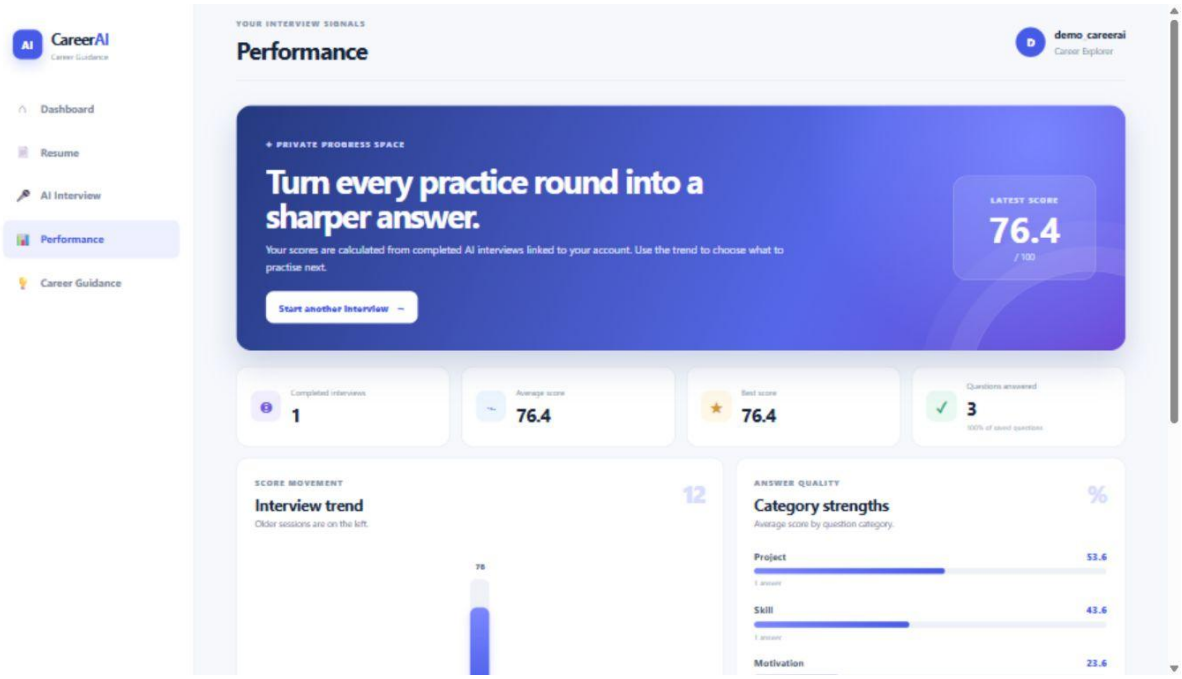


Figure 10.9: Performance Analytics Screen

The performance screen summarizes completed interview sessions for the signed-in user only. It displays latest score, average score, best score, category analysis, and recent interview links.

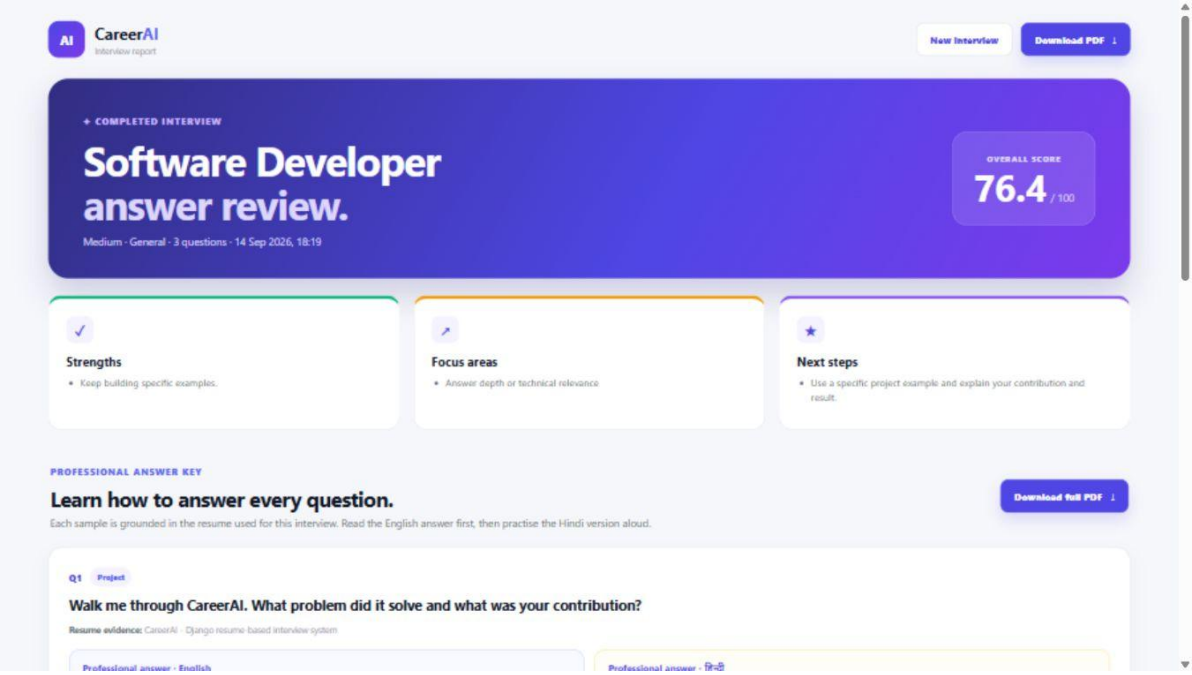


Figure 10.10: Interview Report Screen

The report page displays a completed interview score, coaching feedback, questions, candidate answers, and professional model answers. It also links to PDF report download.

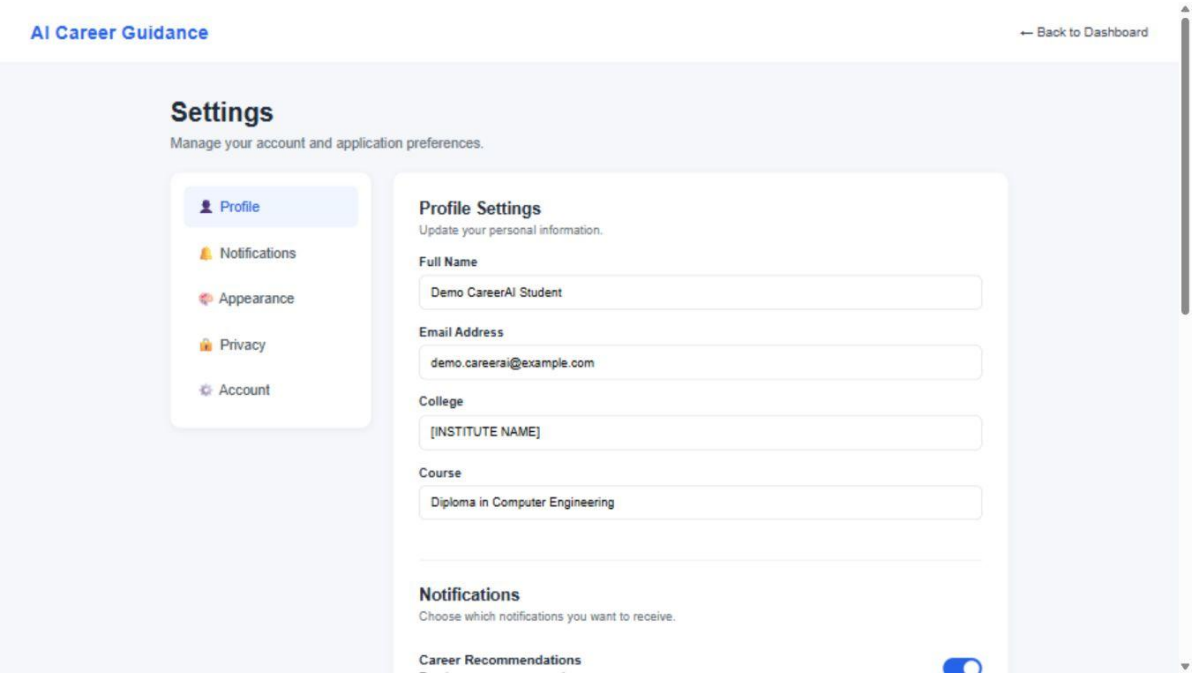


Figure 10.11: User Settings Screen

The settings screen lets the user edit profile information, email, notification settings, theme preference, and profile visibility values stored in Student and UserSettings models.

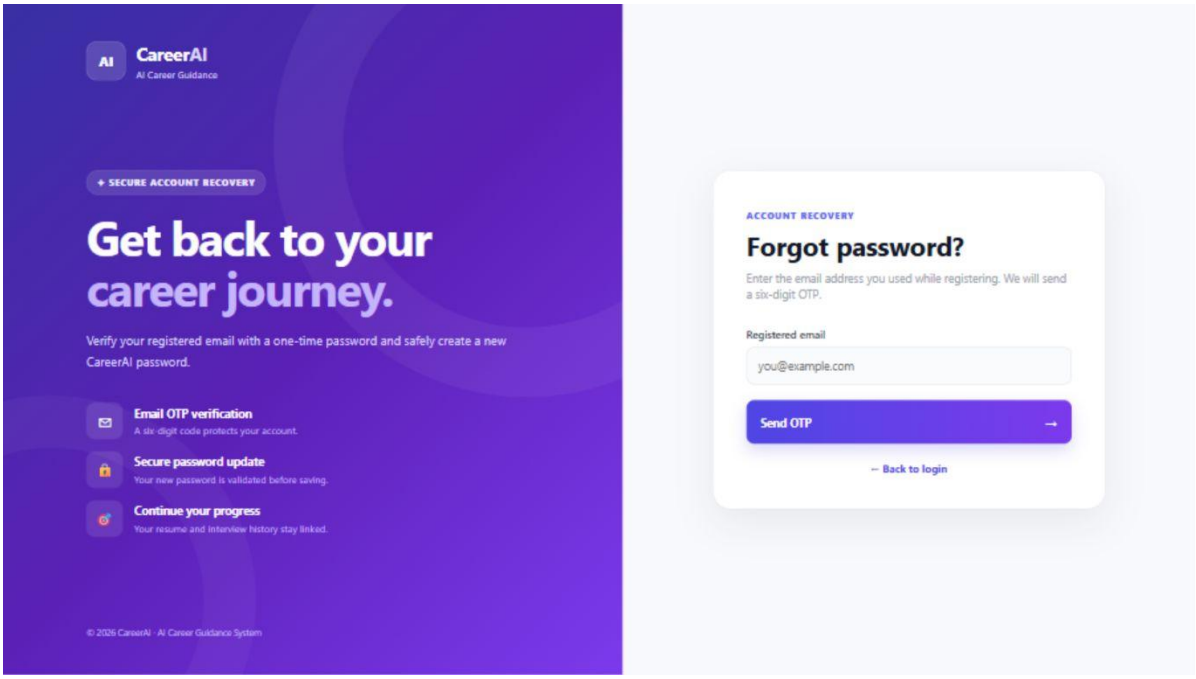


Figure 10.12: Password Recovery Screen

The password recovery page starts the OTP reset workflow. The submitted email is handled with neutral messaging so the system does not reveal whether an account exists.

CHAPTER 11
TESTING

The project includes a Django test file users/tests.py. A fresh test run was performed from the manage.py directory using the project virtual environment. The test suite discovered 9 tests. Eight tests passed and one failed because the test expected generated_by to equal local, while the current implementation returned local-fallback for the offline interview start response.

Metric	Observed Result
Command	..\.venv\Scripts\python.exe manage.py test users
Tests discovered	9
Passed	8
Failed	1
System check	No issues reported
Failure	test_offline_interview_saves_answer_and_report expected local but received local-fallback

Test Case ID	Module	Scenario	Expected Result	Actual Result	Status
AT-01	Authentication	Anonymous dashboard access	Redirect to login	Covered by test and passed	Pass
AT-02	Resume Upload	Upload non-PDF file	Reject file	Covered by test and passed	Pass
AT-03	Resume Processing	Upload PDF and build profile	Resume parse_status complete	Covered by test but contains response-text mismatch risk	Pass in related assertions
AT-04	Interview	Start offline interview	generated_by local	Returned local-fallback	Fail
AT-05	Password Reset	Valid OTP and new password	New password works, old password fails	Covered by test and passed	Pass
AT-06	Performance Privacy	Other user's interview hidden	Private data not shown	Covered by test and passed	Pass

11.9 Proposed Manual Test Cases

ID	Module	Input	Expected Result	Execution Status

MT-01	Registration UI	New username, unique email, strong password	Account created and redirected to login	Proposed manual
MT-02	Resume Upload UI	Scanned image PDF	Readable-text error shown	Proposed manual
MT-03	Career Guidance	POST after resume upload	Target roles and plan displayed	Proposed manual
MT-04	Voice Interview	Browser with speech support	Question audio and transcript controls work	Proposed manual
MT-05	PDF Report	Completed interview	Downloadable PDF opens correctly	Proposed manual
MT-06	Settings	Updated full name, college, course, email	Profile and settings values persist	Proposed manual
MT-07	OTP Resend	Resend before cooldown	Cooldown message appears	Proposed manual
MT-08	OTP Attempts	More than maximum wrong OTP attempts	Reset flow clears and asks user to restart	Proposed manual
MT-09	Interview Count	Hard difficulty	Question count fixed at 40	Proposed manual
MT-10	Interview History API	Authenticated GET request	Recent sessions returned for same user	Proposed manual
MT-11	Report Access Control	Try another user's report URL	Report not shown	Proposed manual
MT-12	Upload Size Limit	PDF larger than RESUME_MAX_SIZE	Upload rejected	Proposed manual
MT-13	Career Guidance Without Resume	POST before upload	Upload resume message shown	Proposed manual
MT-14	Malformed JSON	Invalid body to interview API	400 JSON error	Proposed manual
MT-15	Theme Selection	light, dark, system	Dashboard follows saved theme behavior	Proposed manual

CHAPTER 12
SECURITY

- Django authentication and password hashing are used for account login and password storage.
- CSRF middleware is enabled and templates include csrf_token for forms and interview AJAX requests.
- Protected pages use login_required decorators.
- Password reset stores a hashed OTP, not the plain OTP, and applies expiry, attempt limit, and resend cooldown.
- Successful password reset invalidates existing sessions for the recovered account.
- Forgot-password response text avoids revealing whether an email exists.
- Resume upload checks extension, server-side size limit, and PDF signature before processing.
- Production settings enable secure cookies, HSTS, clickjacking denial, referrer policy, content sniffing protection, and HTTPS redirect when DEBUG is false.
- The project does not implement shared IP/account rate limiting, malware scanning, or OCR sandboxing; these are future hardening items.

CHAPTER 13
RESULTS AND DISCUSSION

The final working system is a Django web application capable of supporting the main CareerAI workflow: authenticated user access, resume upload, profile extraction, career guidance, resume-grounded interview practice, scoring, feedback, performance history, and PDF report generation. The implementation is especially suitable for academic demonstration because the default mode works without an external AI key.

Feature	Verified From Source	Discussion
Authentication	Yes	Registration, login, logout, protected views
Resume Processing	Yes	PDF extraction and local profile generation
Career Guidance	Yes	Rule-based target roles, gaps, project ideas, and 30-day plan
Interview	Yes	Session, questions, answers, scoring, feedback
Performance	Yes	Private analytics from completed interviews
PDF Report	Yes	ReportLab generated PDF
External AI	Optional only	Disabled by default; provider hooks require configuration

CHAPTER 14
ADVANTAGES LIMITATIONS AND FUTURE SCOPE

14.1 Advantages

- Resume-grounded preparation
- No paid API required in default mode
- Django security foundation
- Clear database models
- PDF reports for revision
- Bilingual professional answer key

14.2 Limitations

- No OCR for scanned resumes
- Rule-based analysis has limited semantic understanding
- No production rate limiter
- No malware scanning for uploaded PDFs
- No real job board integration
- No mobile app is included

14.3 Future Enhancements

- Add OCR and stronger resume parsing
- Add cache-backed throttling
- Add cloud deployment configuration
- Add mobile responsive testing and mobile application
- Add advanced AI/ML ranking as a clearly separate feature
- Add job recommendation integration
- Add richer admin dashboard and analytics

CHAPTER 15
CONCLUSION

CareerAI - AI-Powered Career Guidance and Resume-Based Interview System was developed as a Diploma Computer Engineering project using Django, SQLite, Python, HTML, CSS, JavaScript, pypdf, and ReportLab. The system addresses the practical problem of helping students prepare for careers and interviews using their own resume evidence. It includes registration, login, password recovery, settings, resume upload, local profile analysis, career guidance, resume-grounded interviews, answer scoring, feedback, performance tracking, HTML reports, and PDF reports. The project demonstrates useful learning outcomes in web development, authentication, ORM-based database design, file upload validation, JSON-based browser APIs, local text processing, and report generation. Its limitations are clearly identified, and future work can extend it with stronger parsing, rate limiting, OCR, cloud deployment, and job recommendation features.

References

1. Python Software Foundation. Python Documentation. https://docs.python.org/
2. Django Software Foundation. Django Documentation. https://docs.djangoproject.com/
3. SQLite Consortium. SQLite Documentation. https://www.sqlite.org/docs.html
4. Django REST Framework. Official Documentation. https://www.django-rest-framework.org/
5. pypdf Contributors. pypdf Documentation. https://pypdf.readthedocs.io/
6. ReportLab. ReportLab User Guide and Documentation. https://www.reportlab.com/docs/
7. Mozilla Developer Network. HTML, CSS, and JavaScript Web Documentation. https://developer.mozilla.org/

Appendix A - Project Directory Structure

```
C:\Users\SAKSHAM\OneDrive\Documents\Project
|-- Project
|   |-- manage.py
|   |-- db.sqlite3
|   |-- requirements.txt
|   |-- career_guidance
|   |   |-- settings.py
|   |   |-- urls.py
|   |   |-- asgi.py
|   |   |-- wsgi.py
|   |-- users
|       |-- models.py
|       |-- forms.py
|       |-- views.py
|       |-- serializers.py
|       |-- resume_services.py
|       |-- pdf_reports.py
|       |-- tests.py
|       |-- migrations
|       |-- templates/users
```

```
|       |-- static/users
|-- .env.example
|-- SECURITY_AUDIT_20260829.md
|-- README.md
|-- LICENSE
```

Appendix B - User Manual

- Open the application in a browser after running the Django server.
- Register with username, valid unique email address, and strong password.
- Log in from the CareerAI login page.
- Open Settings to update full name, college, course, email, notifications, theme, and profile visibility.
- Open Upload Resume and select a valid text-based PDF resume.
- Review the upload result and confirm that resume text was processed successfully.
- Open Career Guidance and generate suggestions based on the latest resume.
- Open AI Interview, choose role, difficulty, interview type, language, and question count where applicable.
- Start the interview, listen to or read questions, and submit answers.
- Finish the interview to calculate overall score and feedback.
- Open Performance to view score trends and recent completed interviews.
- Open the interview report to review professional English and Hindi answer models.
- Download the PDF report for offline revision.
- Use Forgot Password if account recovery is required.

Appendix C - Installation Instructions

```
cd C:\Users\SAKSHAM\OneDrive\Documents\Project\Project
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver

Optional .env values:
DJANGO_SECRET_KEY=<secure secret>
DJANGO_DEBUG=1
EMAIL_HOST_USER=<email address>
EMAIL_HOST_PASSWORD=<app password>
ENABLE_AI=false
AI_PROVIDER=local
GEMINI_API_KEY=<only if AI_PROVIDER=gemini and ENABLE_AI=true>
```

Appendix D - Important Notes for Submission

- Do not submit real .env secrets, personal API keys, or production passwords.
- Do not publish uploaded resumes from the media folder.

- Replace all cover-page placeholders with institute, guide, student, enrollment, and date details before printing.
- Capture real screenshots from the running application and replace the placeholders in Chapter 10.
- Fix the current automated test expectation or implementation label mismatch before claiming all tests pass.

Appendix E - Detailed Module Inventory

Module	Source Files	Responsibility	Database Interaction
Landing/Home	index.html, index.css, landing.js	Public page with feature descriptions and links to login/register	No database write
Registration	RegisterForm, register view, register.html	Creates Django User after email and password validation	auth_user
Login	user_login, login.html	Authenticates username and password	Django auth session
Password Reset	forgot_password, verify_password_otp, reset_password templates	Sends and verifies OTP, changes password, invalidates sessions	auth_user, django_session
Settings	settings_page, settings.html	Updates profile, email, notification settings, theme, privacy	Student, UserSettings, auth_user
Resume Upload	upload_resume, upload_resume.html	Receives PDF, validates extension, size, header, extracts text	Resume
Resume Processing	resume_services.py	Builds local profile from resume text	Resume.raw_text, Resume.parsed_profile
Career Guidance	career_guidance_page, career_guidance.html	Generates role suggestions, strengths, gaps, projects, plan	Reads Resume and latest InterviewFeedback
Interview UI	ai_interview.html, ai_interview.css	Browser interface with role, difficulty, type, language, question count	Calls JSON APIs
Question API	interview_start_api	Creates interview session and question rows	InterviewSession, InterviewQuestion
Answer API	interview_answer_api	Scores and stores each answer	InterviewAnswer
Finish API	interview_finish_api	Calculates overall score and feedback	InterviewSession, InterviewFeedback
Report Page	interview_report, interview_report.html	Shows question-wise report and answer key	Reads session, questions, answers, feedback
PDF Report	interview_report_pdf, pdf_reports.py	Generates downloadable ReportLab PDF	Reads completed interview data

Performance	performance_page, performance.html	Shows score trend, category scores, recent completed interviews	InterviewSession, InterviewAnswer

Appendix F - Module Input Processing Output Details

Module	Input	Processing	Output
Registration	username, email, password	Validate unique email and password strength; hash password	User account
Login	username and password	Authenticate through Django	Authenticated session or error
Settings	profile, email, checkboxes, theme	Validate email and save related settings	Updated Student, User, UserSettings
Resume Upload	PDF file	Check extension, size and signature; save file; extract text	Resume row with parse status
Profile Builder	raw resume text	Detect skills and sections with regular expressions	parsed_profile JSON
Career Guidance	latest resume profile and optional feedback	Select target roles, gaps, projects, next steps	Guidance displayed in template
Interview Start	role, difficulty, type, language, count	Clamp values; generate questions; bulk create rows	questions JSON
Answer Submit	question_id, answer, duration	Check ownership; calculate scores and feedback	answer score JSON
Interview Finish	session_id	Average answer scores; generate feedback and model answers	report and PDF URLs
Performance	current user	Aggregate completed sessions and category score data	analytics page
PDF Report	completed session id	Build ReportLab document	application/pdf response
OTP Request	email address	Neutral lookup; generate hashed OTP; send email	redirect or notice
OTP Verify	six digit OTP	Check expiry, attempts, hash	verified session state
New Password	password and confirmation	Validate and set password; invalidate sessions	login redirect

Appendix G - Expanded Data Dictionary Notes

Student

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

The field user is stored as OneToOneField User. Student belongs to one user. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

The field full_name is stored as CharField(100). Student name. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

The field college is stored as CharField(100). College name. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

The field course is stored as CharField(100, blank). Course name. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

The field cgpa is stored as FloatField. Academic score. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

The field skills is stored as TextField(blank). Manually entered skills. In the submitted implementation, this field is part of the Student model and is accessed through Django ORM queries rather than direct SQL in application code.

UserSettings

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

The field user is stored as OneToOneField User. Settings owner. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

The field career_notifications is stored as BooleanField. Career notification preference. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

The field resume_notifications is stored as BooleanField. Resume notification preference. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

The field job_notifications is stored as BooleanField. Job notification preference. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

The field theme is stored as CharField choices. light, dark, or system. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

The field profile_visibility is stored as BooleanField. Profile visibility preference. In the submitted implementation, this field is part of the UserSettings model and is accessed through Django ORM queries rather than direct SQL in application code.

Resume

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field user is stored as ForeignKey User. Owner. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field resume_file is stored as FileField. Uploaded under resumes/. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field raw_text is stored as TextField. Extracted PDF text. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field parsed_profile is stored as JSONField. Detected skills, education, projects, experience. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field parse_status is stored as CharField choices. pending, complete, failed. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field parse_error is stored as CharField(255). Extraction error. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

The field uploaded_at is stored as DateTimeField. Upload timestamp. In the submitted implementation, this field is part of the Resume model and is accessed through Django ORM queries rather than direct SQL in application code.

InterviewSession

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field user is stored as ForeignKey User. Owner. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field resume is stored as ForeignKey Resume nullable. Resume evidence. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field role is stored as CharField(100). Target role. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field difficulty is stored as CharField(20). easy, medium, hard. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field interview_type is stored as CharField(30). general, technical, behavioral. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field status is stored as CharField(20). in_progress or completed. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field question_count is stored as PositiveSmallIntegerField. Number of questions. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field overall_score is stored as FloatField nullable. Final score. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field started_at is stored as DateTimeField. Start time. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

The field completed_at is stored as DateTimeField nullable. Finish time. In the submitted implementation, this field is part of the InterviewSession model and is accessed through Django ORM queries rather than direct SQL in application code.

InterviewQuestion

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field session is stored as ForeignKey InterviewSession. Parent session. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field order is stored as PositiveSmallIntegerField. Question order. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field question_text is stored as TextField. Question. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field category is stored as CharField(50). Question category. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field source is stored as CharField(30). local, ai, project, skill, fallback. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field source_evidence is stored as CharField(255). Resume evidence phrase. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

The field expected_topics is stored as JSONField. Expected answer topics. In the submitted implementation, this field is part of the InterviewQuestion model and is accessed through Django ORM queries rather than direct SQL in application code.

InterviewAnswer

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field question is stored as OneToOneField InterviewQuestion. One answer per question. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field answer_text is stored as TextField. Candidate answer. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field duration_seconds is stored as PositiveIntegerField nullable. Optional duration. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field technical_score is stored as FloatField. Technical score. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field relevance_score is stored as FloatField. Relevance score. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field communication_score is stored as FloatField. Communication score. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field overall_score is stored as FloatField. Overall score. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field filler_count is stored as PositiveIntegerField. Detected filler words. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field feedback is stored as JSONField. Feedback items. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

The field submitted_at is stored as DateTimeField. Last submission time. In the submitted implementation, this field is part of the InterviewAnswer model and is accessed through Django ORM queries rather than direct SQL in application code.

InterviewFeedback

The field id is stored as BigAutoField. Primary key. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field session is stored as OneToOneField InterviewSession. Completed session. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field strengths is stored as JSONField. Strength list. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field weaknesses is stored as JSONField. Weakness list. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field recommendations is stored as JSONField. Recommendations. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field professional_answers is stored as JSONField. Bilingual model answers. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field generated_by is stored as CharField(20). local or provider indicator. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

The field created_at is stored as DateTimeField. Generation time. In the submitted implementation, this field is part of the InterviewFeedback model and is accessed through Django ORM queries rather than direct SQL in application code.

Appendix H - Source Code Reference Extracts

H.1 settings.py database and media configuration

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.2 settings.py optional AI configuration

```
AI_PROVIDER = os.getenv('AI_PROVIDER', 'local').lower()
AI_ENABLED = os.getenv('ENABLE_AI', 'false').lower() in {'1', 'true', 'yes', 'on'}
GEMINI_API_KEY = os.getenv('GEMINI_API_KEY', '')
OPENAI_API_KEY = os.getenv('OPENAI_API_KEY', '')
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.3 RegisterForm email validation

```
email = forms.EmailField(required=True)

def clean_email(self):
    email = self.cleaned_data['email'].strip().lower()
    if User.objects.filter(email__iexact=email).exists():
        raise forms.ValidationError('An account with this email already exists.')
    return email
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.4 InterviewSession model summary

```
class InterviewSession(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='interviews')
    resume = models.ForeignKey(Resume, on_delete=models.SET_NULL, null=True, blank=True,
related_name='interviews')
    role = models.CharField(max_length=100)
    difficulty = models.CharField(max_length=20, default='medium')
    interview_type = models.CharField(max_length=30, default='general')
    status = models.CharField(max_length=20, default='in_progress')
    question_count = models.PositiveSmallIntegerField(default=5)
    overall_score = models.FloatField(null=True, blank=True)
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.5 Resume profile builder return format

```
return {
    'skills': skills,
    'education': education,
    'experience': experience,
    'projects': projects,
    'certifications': certifications,
    'source': 'local',
}
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.6 Interview start ownership and resume requirement

```
resume = Resume.objects.filter(user=request.user).order_by('-uploaded_at').first()
if resume is None:
    return JsonResponse({'error': 'Upload a PDF resume before starting an interview.'}, status=400)
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.7 OTP expiry setting usage

```
expiry = max(60, int(getattr(settings, 'PASSWORD_RESET_OTP_EXPIRY_SECONDS', 600)))
return not started_at or (time.time() - started_at) > expiry
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

H.8 Performance privacy query

```
completed_sessions = list(
    InterviewSession.objects.filter(user=request.user, status='completed')
    .exclude(overall_score__isnull=True)
    .order_by('-completed_at', '-started_at')
    .prefetch_related('questions__answer')
)
```

This extract is included to help the examiner connect the report explanation with the actual submitted Django source code.

Appendix I - Extended Manual Test Case Catalogue

ID	Module	Scenario	Expected Result	Status
MTC-01	Registration	Submit blank registration form	Required field errors should be shown	Proposed manual
MTC-02	Registration	Register with duplicate email	Duplicate email validation error should be shown	Proposed manual
MTC-03	Registration	Register with weak numeric password	Django password validator should reject it	Proposed manual
MTC-04	Login	Submit valid credentials	User should be redirected to dashboard	Proposed manual
MTC-05	Login	Submit invalid credentials	Invalid username or password message should appear	Proposed manual
MTC-06	Settings	Change notification checkboxes	Preferences should be saved	Proposed manual
MTC-07	Settings	Use invalid email	Validation message should appear	Proposed manual
MTC-08	Upload	Upload no file	Please select a resume file first should appear	Proposed manual
MTC-09	Upload	Upload .docx file	Only PDF resumes are supported should appear	Proposed manual
MTC-10	Upload	Upload fake PDF extension without PDF header	Invalid PDF message should appear	Proposed manual
MTC-11	Upload	Upload valid text PDF	Resume should be saved and analysed	Proposed manual
MTC-12	Career Guidance	Generate without resume	Upload resume prompt should appear	Proposed manual

MTC-13	Career Guidance	Generate after parsed resume	Target roles and plan should appear	Proposed manual
MTC-14	Interview Start	Start without resume	400 error requiring PDF resume	Proposed manual
MTC-15	Interview Start	Invalid difficulty	Difficulty should default to medium	Proposed manual
MTC-16	Interview Start	Hard difficulty	40 questions should be requested	Proposed manual
MTC-17	Interview Answer	Invalid question id	400 or 404 response should be returned	Proposed manual
MTC-18	Interview Answer	Valid answer	Scores should be returned and saved	Proposed manual
MTC-19	Interview Finish	Invalid session id	404 response should be returned	Proposed manual
MTC-20	Interview Finish	Completed session	Overall score, feedback, report URL and PDF URL should be returned	Proposed manual
MTC-21	Interview Report	Open incomplete session report	User should be redirected to interview page	Proposed manual
MTC-22	Interview Report	Open completed own report	Report should display questions, answer key, and score	Proposed manual
MTC-23	PDF Report	Download before completion	400 message should be returned	Proposed manual
MTC-24	PDF Report	Download after completion	PDF file should be downloaded	Proposed manual
MTC-25	Performance	No completed interviews	Empty state should be displayed	Proposed manual
MTC-26	Performance	Completed interviews exist	Trend and recent sessions should be displayed	Proposed manual
MTC-27	Password Reset	Unknown email	Neutral notice should be displayed	Proposed manual
MTC-28	Password Reset	Correct OTP	User should continue to new password page	Proposed manual
MTC-29	Password Reset	Wrong OTP repeatedly	Attempts should reduce and flow should reset after limit	Proposed manual
MTC-30	Password Reset	Resend too quickly	Cooldown message	Proposed manual

			should be displayed	

Appendix J - Academic Formatting Checklist

- Use A4 paper size before final printing.
- Use Times New Roman or another approved academic font.
- Replace placeholders for institute, guide, student names, enrollment numbers, and date.
- Insert real application screenshots in Chapter 10 after running the server.
- Update the table of contents page numbers in Microsoft Word before printing.
- Keep implemented features and future enhancements separate during viva explanation.
- Explain clearly that default processing is local and rule-based, with optional provider integration only when configured.

Appendix K - Requirements Traceability Matrix

ID	Requirement	Implementation Evidence	Verification	Status
FR-01	Account registration	RegisterForm, register view	users/tests.py registration email test	Implemented
FR-02	Login and logout	user_login, user_logout	Malformed login test	Implemented
FR-03	Protected dashboard	login_required dashboard	Anonymous dashboard redirect test	Implemented
FR-04	Settings management	settings_page, UserSettings, Student	Manual testing proposed	Implemented
FR-05	PDF resume upload	upload_resume	Non-PDF upload test	Implemented
FR-06	PDF text extraction	extract_resume_text	Mock extraction upload test	Implemented
FR-07	Profile analysis	build_local_profile	Local profile assertion	Implemented
FR-08	Career guidance	career_guidance_page	Manual testing proposed	Implemented
FR-09	Interview question generation	generate_questions, interview_start_api	Offline interview test with one label mismatch	Implemented with failing assertion
FR-10	Answer evaluation	evaluate_answer, interview_answer_api	Offline interview answer test	Implemented
FR-11	Interview completion	interview_finish_api	Offline interview finish test	Implemented
FR-12	Performance dashboard	performance_page	Performance privacy	Implemented

			test	
FR-13	HTML report	interview_report	Manual testing proposed	Implemented
FR-14	PDF report	interview_report_pdf, pdf_reports.py	Manual testing proposed	Implemented
FR-15	OTP password reset	forgot_password and related views	Valid OTP reset test	Implemented
NFR-01	Security	settings.py middleware and decorators	Django check and security-oriented tests	Partially implemented
NFR-02	Maintainability	Separate models, views, services, reports	Source inspection	Implemented
NFR-03	Deployability	requirements.txt, .env.example, migrations	Django check	Implemented for local deployment
NFR-04	Privacy	User-filtered queries and media ignored by Git	Performance privacy test	Partially implemented
NFR-05	Reliability	Local fallback around provider calls	Source inspection	Implemented

Appendix L - File by File Source Structure

Path	Purpose
manage.py	Django command entry point used for runserver, migrations, tests, and checks
career_guidance/settings.py	Project settings, installed apps, middleware, SQLite database, static/media paths, email, security and optional provider configuration
career_guidance/urls.py	Root URL router connecting admin and users.urls
career_guidance/asgi.py	ASGI application entry point
career_guidance/wsgi.py	WSGI application entry point
users/models.py	Application data models for settings, student profile, resume, interview sessions, questions, answers, and feedback
users/forms.py	Registration and profile model forms
users/views.py	Main page views, password reset flow, resume upload, career guidance, interview APIs, report views and performance page
users/resume_services.py	Resume text extraction, local profile analysis, optional provider adapter, question generation, scoring and guidance logic

users/pdf_reports.py	ReportLab PDF generation for completed interviews
users/serializers.py	Present but not functionally used for API serialization in current source
users/tests.py	Django TestCase classes for authentication, upload, interview, reset, and privacy behavior
users/templates/users/index.html	Landing page
users/templates/users/register.html	Registration page
users/templates/users/login.html	Login page
users/templates/users/dashboard.html	Authenticated dashboard
users/templates/users/upload_resume.html	Resume upload page
users/templates/users/career_guidance.html	Career suggestion page
users/templates/users/ai_interview.html	Interactive interview page and JavaScript API calls
users/templates/users/interview_report.html	HTML interview report and answer key
users/templates/users/performance.html	Performance analytics page
users/templates/users/settings.html	Profile, notification, theme and privacy settings
users/templates/users/forgot_password.html	Password reset request page
users/templates/users/verify_otp.html	OTP verification page
users/templates/users/reset_password.html	New password page
users/templates/users/emails/password_reset_otp.html	HTML email body for OTP
users/static/users/*.css	Page-specific styling files
users/static/users/landing.js	Landing page animation script
users/static/users/favicon.svg	Site icon
users/migrations/*.py	Database migration history for all application models and fields

Appendix M - Actual Source Code Listings

The following listings are copied from the submitted project source in order to support viva discussion. They are included as reference excerpts and should not be treated as separate invented code.

M.1 users models.py

```
from django.db import models
from django.contrib.auth.models import User
```

```
class UserSettings(models.Model):
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name="settings"
    )

    career_notifications = models.BooleanField(default=True)
    resume_notifications = models.BooleanField(default=True)
    job_notifications = models.BooleanField(default=False)

    theme = models.CharField(
        max_length=10,
        choices=[
            ("light", "Light"),
            ("dark", "Dark"),
            ("system", "System Default"),
        ],
        default="light"
    )

    profile_visibility = models.BooleanField(default=True)

    def __str__(self):
        return f"{self.user.username} Settings"

class Student(models.Model):

    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE
    )

    full_name = models.CharField(
        max_length=100
    )

    college = models.CharField(
        max_length=100
    )

    course = models.CharField(
        max_length=100,
        blank=True
    )

    cgpa = models.FloatField(
        default=0
    )

    skills = models.TextField(
        blank=True
    )

    def __str__(self):
        return self.full_name



class Resume(models.Model):

    user = models.ForeignKey(
        User,
```

```
        on_delete=models.CASCADE
    )

    resume_file = models.FileField(
        upload_to="resumes/"
    )

    # Extracted locally with pypdf and optionally enriched by the configured
    # AI provider. Keeping this data avoids sending the PDF on every request.
    raw_text = models.TextField(blank=True)
    parsed_profile = models.JSONField(default=dict, blank=True)
    parse_status = models.CharField(
        max_length=20,
        choices=[
            ("pending", "Pending"),
            ("complete", "Complete"),
            ("failed", "Failed"),
        ],
        default="pending",
    )
    parse_error = models.CharField(max_length=255, blank=True)

    uploaded_at = models.DateTimeField(
        auto_now_add=True
    )


    def __str__(self):
        return self.user.username


class InterviewSession(models.Model):
    """One practice interview linked to the resume used as evidence."""

    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="interviews")
    resume = models.ForeignKey(
        Resume,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="interviews",
    )
    role = models.CharField(max_length=100)
    difficulty = models.CharField(max_length=20, default="medium")
    interview_type = models.CharField(max_length=30, default="general")
    status = models.CharField(max_length=20, default="in_progress")
    question_count = models.PositiveSmallIntegerField(default=5)
    overall_score = models.FloatField(null=True, blank=True)
    started_at = models.DateTimeField(auto_now_add=True)
    completed_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-started_at"]

    def __str__(self):
        return f"{self.user.username} - {self.role} ({self.started_at:%Y-%m-%d})"


class InterviewQuestion(models.Model):
    session = models.ForeignKey(
        InterviewSession,
        on_delete=models.CASCADE,
        related_name="questions",
```

```
    )
    order = models.PositiveSmallIntegerField()
    question_text = models.TextField()
    category = models.CharField(max_length=50, default="General")
    source = models.CharField(max_length=30, default="fallback")
    source_evidence = models.CharField(max_length=255, blank=True)
    expected_topics = models.JSONField(default=list, blank=True)

    class Meta:
        ordering = ["order"]
        constraints = [
            models.UniqueConstraint(
                fields=["session", "order"],
                name="unique_question_order_per_session",
            )
        ]

    def __str__(self):
        return f"Q{self.order}: {self.question_text[:60]}"


class InterviewAnswer(models.Model):
    question = models.OneToOneField(
        InterviewQuestion,
        on_delete=models.CASCADE,
        related_name="answer",
    )
    answer_text = models.TextField(blank=True)
    duration_seconds = models.PositiveIntegerField(null=True, blank=True)
    technical_score = models.FloatField(default=0)
    relevance_score = models.FloatField(default=0)
    communication_score = models.FloatField(default=0)
    overall_score = models.FloatField(default=0)
    filler_count = models.PositiveIntegerField(default=0)
    feedback = models.JSONField(default=dict, blank=True)
    submitted_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"Answer for {self.question_id} ({self.overall_score:.1f})"


class InterviewFeedback(models.Model):
    session = models.OneToOneField(
        InterviewSession,
        on_delete=models.CASCADE,
        related_name="feedback",
    )
    strengths = models.JSONField(default=list, blank=True)
    weaknesses = models.JSONField(default=list, blank=True)
    recommendations = models.JSONField(default=list, blank=True)
    # Stored once when an interview is completed so the report page and PDF
    # always show the same professional bilingual answer key.
    professional_answers = models.JSONField(default=list, blank=True)
    generated_by = models.CharField(max_length=20, default="local")
    created_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"Feedback for interview {self.session_id}"
```

M.2 users forms.py

```
from django import forms
from django.contrib.auth.models import User
from .models import Student


class RegisterForm(forms.ModelForm):

    # Password recovery is email-based, so every newly-created account must
    # provide a usable and unambiguous address.
    email = forms.EmailField(required=True)

    password = forms.CharField(
        widget=forms.PasswordInput
    )

    class Meta:

        model = User

        fields = [
            "username",
            "email",
            "password"
        ]

    def clean_email(self):
        email = self.cleaned_data["email"].strip().lower()
        if User.objects.filter(email__iexact=email).exists():
            raise forms.ValidationError("An account with this email already exists.")
        return email



class ProfileForm(forms.ModelForm):

    class Meta:

        model = Student

        fields = [
            "full_name",
            "college",
            "cgpa",
            "skills"
        ]
```

M.3 users urls.py

```
from django.urls import path
from django.views.generic import RedirectView

from . import views


urlpatterns = [
    path("favicon.ico", RedirectView.as_view(url="/static/users/favicon.svg", permanent=False),
name="favicon"),
    path("", views.landing_page, name="home"),
    path("settings/", views.settings_page, name="settings"),
    path("register/", views.register, name="register"),
    path("login/", views.user_login, name="login"),
    path("forgot-password/", views.forgot_password, name="forgot_password"),
    path("forgot-password/verify/", views.verify_password_otp, name="verify_password_otp"),
    path("forgot-password/resend/", views.resend_password_otp, name="resend_password_otp"),
    path("forgot-password/new/", views.password_reset_new, name="password_reset_new"),
    path("logout/", views.user_logout, name="logout"),
    path("dashboard/", views.dashboard, name="dashboard"),
    path("performance/", views.performance_page, name="performance"),
    path("career-guidance/", views.career_guidance_page, name="career_guidance"),
    path("upload/", views.upload_resume, name="upload_resume"),
    path("ai-interview/", views.ai_interview_page, name="ai_interview"),
    path("interview/<int:session_id>/report/", views.interview_report, name="interview_report"),
    path("interview/<int:session_id>/report.pdf", views.interview_report_pdf, name="interview_report_pdf"),
    path("api/interview/start/", views.interview_start_api, name="interview_start_api"),
    path("api/interview/answer/", views.interview_answer_api, name="interview_answer_api"),
    path("api/interview/finish/", views.interview_finish_api, name="interview_finish_api"),
    path("api/interview/history/", views.interview_history_api, name="interview_history_api"),
]
```

M.4 career_guidance settings.py

```
"""
Django settings for career_guidance project.

Generated by 'django-admin startproject' using Django 6.0.8.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

import os
from pathlib import Path
from django.core.exceptions import ImproperlyConfigured

try:
    from dotenv import load_dotenv
except ImportError:  # .env loading is optional for the offline mode
    load_dotenv = None

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

if load_dotenv:
    load_dotenv(BASE_DIR.parent / ".env")
    # Also support a .env placed beside manage.py, which is the usual layout
    # when this project is copied out of the repository.
    load_dotenv(BASE_DIR / ".env")


# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.getenv(
    "DJANGO_SECRET_KEY",
    "careerai-local-development-secret-change-before-deployment",
)

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = os.getenv("DJANGO_DEBUG", "1").lower() in {"1", "true", "yes", "on"}

if not DEBUG and SECRET_KEY == "careerai-local-development-secret-change-before-deployment":
    raise ImproperlyConfigured("Set DJANGO_SECRET_KEY before running CareerAI with DJANGO_DEBUG=0.")

ALLOWED_HOSTS = [
    host.strip()
    for host in os.getenv("DJANGO_ALLOWED_HOSTS", "localhost,127.0.0.1").split(",")
    if host.strip()
]


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',

    # Third-party apps
    'rest_framework',

    # Local apps
    'users',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

# Safe defaults for both local and hosted deployments.  Secure-only cookie
# flags are enabled automatically when DEBUG is disabled; localhost remains
# usable over plain HTTP during development.
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Lax"
CSRF_COOKIE_SAMESITE = "Lax"
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = "DENY"
SECURE_REFERRER_POLICY = "same-origin"
SECURE_SSL_REDIRECT = os.getenv(
    "SECURE_SSL_REDIRECT",
    "true" if not DEBUG else "false",
).lower() in {"1", "true", "yes", "on"}

if not DEBUG:
    SESSION_COOKIE_SECURE = True
    CSRF_COOKIE_SECURE = True
    SECURE_HSTS_SECONDS = int(os.getenv("SECURE_HSTS_SECONDS", "31536000"))
    SECURE_HSTS_INCLUDE_SUBDOMAINS = True
    SECURE_HSTS_PRELOAD = True

ROOT_URLCONF = 'career_guidance.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'career_guidance.wsgi.application'


# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases
```

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}


# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]


# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/

STATIC_URL = 'static/'


MEDIA_URL = "/media/"

MEDIA_ROOT = BASE_DIR / "media"

# Authentication defaults. Without this, Django redirects protected pages to
# /accounts/login/, which is not a route in this project.
LOGIN_URL = "/login/"
LOGIN_REDIRECT_URL = "/dashboard/"
LOGOUT_REDIRECT_URL = "/login/"

# Password-reset email. With no SMTP credentials the console backend keeps
# local development free: the OTP is printed in the server terminal. For a
# real inbox, set the Gmail SMTP values in .env (use a Gmail App Password).
EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER", "")
EMAIL_HOST_PASSWORD = os.getenv("EMAIL_HOST_PASSWORD", "")
EMAIL_BACKEND = os.getenv(
    "EMAIL_BACKEND",
    "django.core.mail.backends.smtp.EmailBackend"
```

```
    if EMAIL_HOST_USER and EMAIL_HOST_PASSWORD
    else "django.core.mail.backends.console.EmailBackend",
)
EMAIL_HOST = os.getenv("EMAIL_HOST", "smtp.gmail.com")
EMAIL_PORT = int(os.getenv("EMAIL_PORT", "587"))
EMAIL_USE_TLS = os.getenv("EMAIL_USE_TLS", "true").lower() in {"1", "true", "yes", "on"}
DEFAULT_FROM_EMAIL = os.getenv("DEFAULT_FROM_EMAIL", "").strip() or EMAIL_HOST_USER or "no-
reply@careerai.local"
PASSWORD_RESET_OTP_EXPIRY_SECONDS = int(os.getenv("PASSWORD_RESET_OTP_EXPIRY_SECONDS", "600"))
PASSWORD_RESET_MAX_ATTEMPTS = int(os.getenv("PASSWORD_RESET_MAX_ATTEMPTS", "5"))
PASSWORD_RESET_RESEND_COOLDOWN_SECONDS = int(os.getenv("PASSWORD_RESET_RESEND_COOLDOWN_SECONDS", "30"))
REPORT_PDF_FONT_PATH = os.getenv("REPORT_PDF_FONT_PATH", "").strip()

# The interview remains usable without any external service. These values only
# enable the optional provider adapter when the user explicitly configures it.
AI_PROVIDER = os.getenv("AI_PROVIDER", "local").lower()
AI_ENABLED = os.getenv("ENABLE_AI", "false").lower() in {"1", "true", "yes", "on"}
AI_TIMEOUT_SECONDS = int(os.getenv("AI_TIMEOUT_SECONDS", "20"))
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
OPENAI_BASE_URL = os.getenv("OPENAI_BASE_URL", "https://api.openai.com/v1")
OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
OPENROUTER_HTTP_REFERER = os.getenv("OPENROUTER_HTTP_REFERER", "")
OPENROUTER_TITLE = os.getenv("OPENROUTER_TITLE", "CareerAI Resume Interview")

# Resume uploads are deliberately constrained on the server, independent of
# the browser's accept attribute.
RESUME_MAX_SIZE = int(os.getenv("RESUME_MAX_SIZE", str(10 * 1024 * 1024)))
```

M.5 users resume_services.py excerpt

```
"""Resume processing, offline question generation, and optional AI helpers.

The local path is the product baseline. External providers are deliberately
optional: a missing key, timeout, invalid JSON, or quota error never prevents a
user from practising an interview.
"""

from __future__ import annotations

import json
import secrets
import re
from collections import OrderedDict
from typing import Any

from django.conf import settings


class ResumeProcessingError(Exception):
    """Raised when a resume cannot be safely converted to text."""


SKILL_CATALOG = (
    "Python", "Java", "JavaScript", "TypeScript", "C", "C++", "C#",
    "Django", "Flask", "FastAPI", "React", "Angular", "Node.js",
    "HTML", "CSS", "Bootstrap", "REST API", "GraphQL", "SQL", "MySQL",
    "SQLite", "PostgreSQL", "MongoDB", "Git", "GitHub", "Docker",
    "AWS", "Azure", "Linux", "Machine Learning", "Deep Learning",
    "TensorFlow", "PyTorch", "Pandas", "NumPy", "Power BI", "Excel",
    "Figma", "Firebase", "Web Speech API", "OpenAI", "Gemini",
)

SECTION_ALIASES = OrderedDict(
    [
        ("skills", ("skills", "technical skills", "skill set", "technologies")),
        ("education", ("education", "academic", "qualification")),
        ("experience", ("experience", "employment", "work history", "internship")),
        ("projects", ("projects", "project work", "personal projects")),
        ("certifications", ("certifications", "certificates", "courses")),
    ]
)


def extract_resume_text(file_field) -> str:
    """Extract text from a PDF FileField using the local pypdf package."""

    try:
        from pypdf import PdfReader
    except ImportError as exc:  # pragma: no cover - dependency is in requirements
        raise ResumeProcessingError("pypdf is not installed") from exc

    try:
        with file_field.open("rb") as handle:
            reader = PdfReader(handle)
            pages = [(page.extract_text() or "").strip() for page in reader.pages]
    except Exception as exc:
        raise ResumeProcessingError(f"Could not read the PDF: {exc}") from exc

    text = "\n".join(page for page in pages if page).strip()
    if not text:
        raise ResumeProcessingError(
```

```
            "No selectable text was found. Upload a text-based PDF or type your profile manually."
        )
    return text[:100_000]


def _clean_line(value: str) -> str:
    value = re.sub(r"^[\s•●■‣*-]+", "", value)
    return re.sub(r"\s+", " ", value).strip(" -—:")


def _section_lines(text: str) -> dict[str, list[str]]:
    lines = [_clean_line(line) for line in text.splitlines()]
    sections: dict[str, list[str]] = {name: [] for name in SECTION_ALIASES}
    current: str | None = None
    for line in lines:
        if not line:
            continue
        lower = re.sub(r"[^a-z ]", "", line.lower()).strip()
        found = next(
            (name for name, aliases in SECTION_ALIASES.items()
             if lower in aliases or any(lower.startswith(alias + " ") for alias in aliases)),
            None,
        )
        if found:
            current = found
            continue
        if current:
            sections[current].append(line)
    return sections


def _unique(values: list[str]) -> list[str]:
    seen: set[str] = set()
    result: list[str] = []
    for value in values:
        value = value.strip()
        key = value.casefold()
        if value and key not in seen:
            seen.add(key)
            result.append(value)
    return result


def _contains_skill(text: str, skill: str) -> bool:
    """Match a skill as a token, avoiding C matching Computer or JavaScript."""

    return bool(re.search(rf"(?<![A-Za-z0-9]){re.escape(skill)}(?![A-Za-z0-9])", text, re.IGNORECASE))


def build_local_profile(text: str, student=None) -> dict[str, Any]:
    """Create a conservative, deterministic profile without any API call."""

    text = text or ""
    lower = text.casefold()
    skills = [skill for skill in SKILL_CATALOG if _contains_skill(text, skill)]
    if student is not None and getattr(student, "skills", ""):
        skills.extend(re.split(r"[,;/|\n]", student.skills))
    skills = _unique(skills)

    sections = _section_lines(text)
    education = _unique(sections["education"][:8])
    experience = _unique(sections["experience"][:8])
    raw_projects = _unique(sections["projects"][:8])
```

```
    certifications = _unique(sections["certifications"][:8])

    # Resumes often omit headings. Keep only lines that explicitly provide
    # project evidence rather than inventing a project name.
    if not raw_projects:
        raw_projects = _unique(
            [
                _clean_line(line)
                for line in text.splitlines()
                if "project" in line.casefold()
            ][:8]
        )

    projects = []
    for line in raw_projects:
        technologies = [skill for skill in skills if _contains_skill(line, skill)]
        projects.append(
            {
                "name": line[:100],
                "description": line[:300],
                "technologies": technologies,
            }
        )

    return {
        "skills": skills,
        "education": education,
        "experience": experience,
        "projects": projects,
        "certifications": certifications,
        "source": "local",
    }


def _profile_evidence(profile: dict[str, Any]) -> list[str]:
    evidence: list[str] = list(profile.get("skills", []) or [])
    for key in ("education", "experience", "certifications"):
        evidence.extend(profile.get(key, []) or [])
    for project in profile.get("projects", []) or []:
        if isinstance(project, dict):
            evidence.append(str(project.get("name", "")))
            evidence.append(str(project.get("description", "")))
    return [item.strip() for item in evidence if str(item).strip()]


def local_questions(
    profile: dict[str, Any],
    role: str,
    difficulty: str = "medium",
    interview_type: str = "general",
    count: int = 5,
    language: str = "en",
) -> list[dict[str, Any]]:
    """Generate grounded questions from resume evidence, entirely offline."""

    difficulty = difficulty if difficulty in {"easy", "medium", "hard"} else "medium"
    interview_type = interview_type if interview_type in {"general", "technical", "behavioral"} else "general"
    language = language if language in {"en", "hi", "bilingual"} else "en"
    try:
        target_count = max(1, min(int(count), 40))
    except (TypeError, ValueError):
        target_count = 15
    questions: list[dict[str, Any]] = []
```

```
    def localized(english: str, hindi: str) -> str:
        if language == "hi":
            return hindi
        if language == "bilingual":
            return f"{english}\n{hindi}"
        return english

    def add(text: str, category: str, source: str, evidence: str, topics: list[str]):
        questions.append(
            {
                "question": text,
                "type": category,
                "category": category,
                "source": source,
                "source_evidence": evidence[:255],
                "expected_topics": _unique(topics),
            }
        )

    projects = profile.get("projects", []) or []
    skills = profile.get("skills", []) or []
    education = profile.get("education", []) or []
    experience = profile.get("experience", []) or []

    if interview_type in {"general", "technical"}:
        for project in projects[:2]:
            if isinstance(project, dict):
                name = project.get("name") or "the project listed on your resume"
                description = project.get("description") or name
                technologies = project.get("technologies", []) or []
            else:
                name, description, technologies = str(project), str(project), []
            add(
                localized(
                    f"Walk me through {name}. What problem did it solve and what was your contribution?",
                    f"{name} प्रोजेक्ट के बारे में बताइए। इसने कौन-सी समस्या हल की और आपका योगदान क्या था?",
                ),
                "Project",
                "project",
                description,
                ["problem", "contribution", "implementation", *technologies],
            )
            if difficulty != "easy":
                add(
                    localized(
                        f"What technical decision did you make in {name}, and what would you improve now?",
                        f"{name} में आपने कौन-सा तकनीकी निर्णय लिया था और आज आप इसमें क्या सुधार करेंगे?",
                    ),
                    "Project Technical",
                    "project",
                    description,
                    ["decision", "trade-off", "improvement", *technologies],
                )

        for skill in skills[:3]:
            add(
                localized(
                    f"You listed {skill} on your resume. Explain how you have used it and give one concrete
example.",
                    f"आपने रिज्यूमे में {skill} लिखा है। आपने इसका उपयोग कैसे किया? एक वास्तविक उदाहरण दीजिए।",
                ),
                "Skill",
```

```
                "skill",
                skill,
                [skill, "example", "implementation"],
            )

    if interview_type in {"general", "behavioral"}:
        for item in experience[:1]:
            add(
                localized(
                    f"Tell me about the experience described here: {item[:160]}. What did you learn?",
                    f"यहाँ दिए गए अनुभव के बारे में बताइए: {item[:160]} आपने इससे क्या सीखा?",
                ),
                "Experience",
                "experience",
                item,
                ["responsibility", "result", "learning"],
            )
        for item in education[:1]:
            add(
                localized(
                    f"How has your education or coursework prepared you for a {role} position?",
                    f"आपकी शिक्षा या coursework ने आपको {role} भूमिका के लिए कैसे तैयार किया?",
                ),
                "Education",
                "education",
                item,
                ["coursework", "foundation", role],
            )
        add(
            localized(
                f"Why are you interested in working as a {role}, and which resume experience supports that
choice?",
                f"आप {role} के रूप में काम करने में रुचि क्यों रखते हैं और रिज्यूमे का कौन-सा अनुभव इसका समर्थन करता है?",
            ),
            "Motivation",
            "profile",
            ", ".join(_profile_evidence(profile)[:3]) or role,
            [role, "motivation", "experience"],
        )

    if not questions:
        add(
            localized(
                f"What makes you a good candidate for a {role} role? Use only the skills and experience in your
profile.",
                f"आप {role} भूमिका के लिए अच्छे उम्मीदवार क्यों हैं? अपने प्रोफाइल की skills और experience का उपयोग
करें।",
            ),
            "General",
            "fallback",
            role,
            [role, "skills", "experience"],
        )

    # Small or sparse resumes still get a complete practice set. These prompts
    # refer to the user's profile without inventing a technology or employer.
    profile_evidence = ", ".join(_profile_evidence(profile)[:3]) or role
    generic = [
        (
            localized(
                f"Which part of your profile ({profile_evidence[:120]}) best prepares you for this role?",
                f"आपके प्रोफाइल का कौन-सा भाग ({profile_evidence[:120]}) आपको इस भूमिका के लिए सबसे अच्छी तरह तैयार
```

```
करता है?",
            ),
            "Profile",
            ["profile", role, "preparation"],
        ),
        (
            localized(
                "Describe a challenge you faced while learning or building something listed in your profile.",
                "अपने प्रोफाइल में दी गई किसी skill या project को सीखते या बनाते समय आई चुनौती बताइए।",
            ),
            "Behavioral",
            ["challenge", "action", "result"],
        ),
        (
            localized(
                "What would you improve next in the work or project experience shown on your resume?",
                "रिज्यूमे में दिखाए गए work या project experience में आप अगला सुधार क्या करेंगे?",
            ),
            "Improvement",
            ["improvement", "reason", "plan"],
        ),
        (
            localized(
                "Choose one item from your profile and explain how you would measure a successful outcome.",
                "अपने प्रोफाइल की एक item चुनिए और बताइए कि successful outcome को कैसे measure करेंगे।",
            ),
            "Results",
            ["measurement", "outcome", "result"],
        ),
        (
            localized(
                "How would you explain one project or skill from your resume to a non-technical teammate?",
                "अपने रिज्यूमे के किसी project या skill को non-technical teammate को कैसे समझाएंगे?",
            ),
            "Communication",
            ["explanation", "audience", "clarity"],
        ),
    ]
    for text, category, topics in generic:
        if len(questions) >= target_count:
            break
        add(text, category, "profile", profile_evidence, topics)

    # Expand sparse resumes into a realistic long-form interview. Every item
    # still points to an exact resume/profile evidence phrase; only the angle
    # of the question changes.
    evidence_items = _unique(_profile_evidence(profile)) or [role]
    variant_templates = [
        (
            "Explain the most important idea behind {evidence} and how you would teach it to a teammate.",
            "{evidence} के पीछे का सबसे महत्वपूर्ण idea समझाइए और इसे teammate को कैसे सिखाएंगे?",
            "Concept clarity",
            ["explain", "concept", "teammate"],
... excerpt limited to first 360 lines for report readability ...
```

M.6 users views.py excerpt

```
import json
import os
import secrets
import time

from django.shortcuts import render, redirect

from django.contrib.auth.models import User

from django.contrib.auth import authenticate, login, logout
from django.contrib import messages
from django.contrib.auth.hashers import check_password, make_password
from django.contrib.sessions.models import Session

from .forms import RegisterForm

from django.contrib.auth.decorators import login_required
from django.contrib.auth.password_validation import validate_password
from django.core.exceptions import ValidationError
from django.core.mail import EmailMultiAlternatives
from django.core.validators import validate_email
from django.http import HttpResponse, JsonResponse
from django.template.loader import render_to_string
from django.urls import reverse
from django.utils import timezone
from django.views.decorators.http import require_GET, require_POST

from django.shortcuts import render, redirect

from django.conf import settings

from .models import (
    InterviewAnswer,
    InterviewFeedback,
    InterviewQuestion,
    InterviewSession,
    Resume,
    Student,
    UserSettings,
)
from .pdf_reports import build_interview_report_pdf
from .resume_services import (
    ResumeProcessingError,
    build_local_profile,
    evaluate_answer,
    feedback_from_answers,
    generate_career_guidance,
    generate_interview_feedback,
    generate_professional_answers,
    generate_questions,
    extract_resume_text,
)


def landing_page(request):
    """Public first page explaining CareerAI before authentication."""

    return render(request, "users/index.html")
```

```
@login_required
def settings_page(request):

    # Get or create settings for this user
    user_settings, created = UserSettings.objects.get_or_create(
        user=request.user
    )

    # Get or create student profile
    student, created = Student.objects.get_or_create(
        user=request.user,
        defaults={
            "full_name": request.user.get_full_name() or request.user.username,
            "college": "",
            "course": "",
            "cgpa": 0,
            "skills": "",
        }
    )

    # ==========================
    # SAVE SETTINGS
    # ==========================

    if request.method == "POST":

        # -------------------------
        # Profile
        # -------------------------

        full_name = request.POST.get(
            "full_name",
            ""
        ).strip()

        college = request.POST.get(
            "college",
            ""
        ).strip()

        course = request.POST.get(
            "course",
            ""
        ).strip()

        email = request.POST.get(
            "email",
            ""
        ).strip().lower()

        if email:
            try:
                validate_email(email)
            except ValidationError:
                messages.error(request, "Enter a valid email address.")
                return redirect("settings")
            if User.objects.filter(email__iexact=email).exclude(pk=request.user.pk).exists():
                messages.error(request, "That email address is already linked to another account.")
                return redirect("settings")


        # Save profile information
        student.full_name = full_name
```

```
        student.college = college
        student.course = course
        student.save()


        # Save email
        request.user.email = email
        request.user.save()


        # -------------------------
        # Notifications
        # -------------------------

        user_settings.career_notifications = (
            "career_notifications" in request.POST
        )

        user_settings.resume_notifications = (
            "resume_notifications" in request.POST
        )

        user_settings.job_notifications = (
            "job_notifications" in request.POST
        )


        # -------------------------
        # Theme
        # -------------------------

        user_settings.theme = request.POST.get(
            "theme",
            "light"
        )


        # -------------------------
        # Privacy
        # -------------------------

        user_settings.profile_visibility = (
            "profile_visibility" in request.POST
        )


        # Save UserSettings
        user_settings.save()


        # Return to settings page
        return redirect("settings")


    # =========================
    # DISPLAY SETTINGS PAGE
    # =========================

    return render(
        request,
        "users/settings.html",
        {
            "user_settings": user_settings,
```

```
            "student": student,
        }
    )


@login_required
def upload_resume(request):

    user_settings, created = UserSettings.objects.get_or_create(
        user=request.user
    )

    if request.method == "POST":

        resume = request.FILES.get("resume")

        if not resume:
            return render(
                request,
                "users/upload_resume.html",
                {
                    "user_settings": user_settings,
                    "error": "Please select a resume file first."
                }
            )

        extension = os.path.splitext(resume.name or "")[1].lower()
        if extension != ".pdf":
            return render(
                request,
                "users/upload_resume.html",
                {"user_settings": user_settings, "error": "Only PDF resumes are supported."},
            )

        max_size = getattr(settings, "RESUME_MAX_SIZE", 10 * 1024 * 1024)
        if resume.size > max_size:
            return render(
                request,
                "users/upload_resume.html",
                {
                    "user_settings": user_settings,
                    "error": f"Resume must be smaller than {max_size // (1024 * 1024)} MB.",
                },
            )

        # Extension checks alone are trivial to bypass, so also verify the PDF
        # signature before saving user-controlled content.
        header = resume.read(5)
        resume.seek(0)
        if header != b"%PDF-":
            return render(
                request,
                "users/upload_resume.html",
                {"user_settings": user_settings, "error": "The uploaded file is not a valid PDF."},
            )

        saved_resume = Resume.objects.create(
            user=request.user,
            resume_file=resume,
            parse_status="pending",
        )
        try:
            raw_text = extract_resume_text(saved_resume.resume_file)
```

```
            student = Student.objects.filter(user=request.user).first()
            saved_resume.raw_text = raw_text
            saved_resume.parsed_profile = build_local_profile(raw_text, student)
            saved_resume.parse_status = "complete"
            saved_resume.parse_error = ""
            saved_resume.save(update_fields=["raw_text", "parsed_profile", "parse_status", "parse_error"])
            message = "Resume uploaded and locally analysed. You can now start a resume-based interview."
        except ResumeProcessingError as exc:
            saved_resume.parse_status = "failed"
            saved_resume.parse_error = str(exc)[:255]
            saved_resume.save(update_fields=["parse_status", "parse_error"])
            message = f"Resume uploaded, but text extraction failed: {exc}"

        return render(
            request,
            "users/upload_resume.html",
            {
                "user_settings": user_settings,
                "message": message,
                "resume": saved_resume,
            },
        )

    return render(
        request,
        "users/upload_resume.html",
        {
            "user_settings": user_settings
        }
    )


PASSWORD_RESET_SESSION_KEYS = (
    "password_reset_email",
    "password_reset_otp_hash",
    "password_reset_sent_at",
    "password_reset_attempts",
    "password_reset_verified",
    "password_reset_verified_at",
)


def _clear_password_reset_session(request):
    for key in PASSWORD_RESET_SESSION_KEYS:
        request.session.pop(key, None)
    request.session.modified = True


def _password_reset_user(request):
    email = str(request.session.get("password_reset_email", "")).strip().lower()
    if not email:
        return None
    matches = list(User.objects.filter(email__iexact=email, is_active=True).order_by("id")[:2])
    # An address must map to exactly one active account before it can reset a
    # password.  This prevents an old duplicate-email record from resetting
    # the wrong user.
    return matches[0] if len(matches) == 1 else None


def _invalidate_user_sessions(user):
    """Sign out every existing browser session for a user after a reset.

    Django password hashes stop the old password from authenticating, but a
```

```
    browser that was already signed in can otherwise keep using its existing
    session.  Removing those sessions makes the reset behaviour unambiguous
    and is the safer recovery policy.
    """

    user_id = str(user.pk)
    sessions = Session.objects.filter(expire_date__gt=timezone.now())
    for session in sessions.iterator():
        try:
            session_data = session.get_decoded()
        except (KeyError, TypeError, ValueError):
            # Ignore malformed/stale rows rather than breaking a password
            # reset for the account being recovered.
            continue
        if str(session_data.get("_auth_user_id", "")) == user_id:
            session.delete()


def _password_reset_expired(request, timestamp_key="password_reset_sent_at"):
    try:
        started_at = float(request.session.get(timestamp_key, 0))
    except (TypeError, ValueError):
        return True
    expiry = max(60, int(getattr(settings, "PASSWORD_RESET_OTP_EXPIRY_SECONDS", 600)))
    return not started_at or (time.time() - started_at) > expiry


def _mask_email(email):
    local, separator, domain = email.partition("@")
    if not separator:
        return email
    if len(local) <= 2:
        masked_local = local[:1] + "*"
    else:
        masked_local = f"{local[0]}{'*' * max(1, len(local) - 2)}{local[-1]}"
    return f"{masked_local}@{domain}"


def _send_password_reset_otp(request, user):
    """Create a short-lived hashed OTP and send it through configured email."""

    otp = f"{secrets.randbelow(1_000_000):06d}"
    email = user.email.strip().lower()
    request.session["password_reset_email"] = email
    request.session["password_reset_otp_hash"] = make_password(otp)
    request.session["password_reset_sent_at"] = int(time.time())
    request.session["password_reset_attempts"] = 0
... excerpt limited to first 360 lines for report readability ...
```

M.7 users pdf_reports.py

```
"""Polished, resume-grounded interview report PDFs."""

from __future__ import annotations

import html
import os
from io import BytesIO
from pathlib import Path
from typing import Any

from django.conf import settings


def _font_paths() -> list[Path]:
    base_dir = Path(getattr(settings, "BASE_DIR", Path.cwd()))
    windows_dir = Path(str(Path.home()))
    windir = Path(str(os.environ.get("WINDIR", "C:/Windows")))
    paths = [
        base_dir / "users" / "static" / "users" / "fonts" / "Nirmala.ttf",
        base_dir / "users" / "static" / "users" / "fonts" / "NirmalaUI.ttf",
        base_dir / "users" / "static" / "users" / "fonts" / "Mangal.ttf",
        base_dir / "static" / "fonts" / "NotoSansDevanagari-Regular.ttf",
        windir / "Fonts" / "Nirmala.ttf",
        windir / "Fonts" / "NirmalaUI.ttf",
        windir / "Fonts" / "Mangal.ttf",
        windows_dir / "AppData" / "Local" / "Microsoft" / "Windows" / "Fonts" / "Nirmala.ttf",
        Path("/usr/share/fonts/truetype/noto/NotoSansDevanagari-Regular.ttf"),
        Path("/usr/share/fonts/truetype/lohit-devanagari/Lohit-Devanagari.ttf"),
    ]
    configured_path = str(getattr(settings, "REPORT_PDF_FONT_PATH", "")).strip()
    if configured_path:
        paths.insert(0, Path(configured_path))
    return paths


def _register_fonts():
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont

    regular = "Helvetica"
    bold = "Helvetica-Bold"
    for font_path in _font_paths():
        if not font_path.is_file():
            continue
        try:
            pdfmetrics.registerFont(TTFont("CareerAIUnicode", str(font_path)))
            regular = "CareerAIUnicode"
            bold = "CareerAIUnicode"
            break
        except Exception:
            continue
    return regular, bold


def _paragraph_text(value: Any) -> str:
    text = html.escape(str(value or "").strip())
    return text.replace("\r\n", "\n").replace("\n", "<br/>") or "-"


def _list_items(items: Any, limit: int = 6) -> list[str]:
    if not isinstance(items, list):
```

```
        return []
    result = []
    for item in items[:limit]:
        text = str(item or "").strip()
        if text:
            result.append(text)
    return result


def _answer_map(feedback) -> dict[int, dict[str, Any]]:
    values = getattr(feedback, "professional_answers", []) or []
    if not isinstance(values, list):
        return {}
    result = {}
    for item in values:
        if not isinstance(item, dict):
            continue
        try:
            result[int(item.get("order"))] = item
        except (TypeError, ValueError):
            continue
    return result


def build_interview_report_pdf(session) -> bytes:
    """Return a complete PDF report for one completed InterviewSession."""

    try:
        from reportlab.lib import colors
        from reportlab.lib.enums import TA_CENTER, TA_LEFT
        from reportlab.lib.pagesizes import A4
        from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
        from reportlab.lib.units import mm
        from reportlab.platypus import (
            KeepTogether,
            PageBreak,
            Paragraph,
            SimpleDocTemplate,
            Spacer,
            Table,
            TableStyle,
        )
    except ImportError as exc:  # pragma: no cover - dependency is declared in requirements
        raise RuntimeError("reportlab is not installed. Run pip install -r requirements.txt") from exc

    regular_font, bold_font = _register_fonts()
    purple = colors.HexColor("#4f46e5")
    dark_purple = colors.HexColor("#312e81")
    ink = colors.HexColor("#111827")
    muted = colors.HexColor("#6b7280")
    light_purple = colors.HexColor("#f5f3ff")
    line = colors.HexColor("#e5e7eb")

    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        "CareerAITitle",
        parent=styles["Title"],
        fontName=bold_font,
        fontSize=25,
        leading=30,
        textColor=ink,
        alignment=TA_LEFT,
        spaceAfter=6,
```

```
)
subtitle_style = ParagraphStyle(
    "CareerAISubtitle",
    parent=styles["Normal"],
    fontName=regular_font,
    fontSize=10,
    leading=15,
    textColor=muted,
    spaceAfter=16,
)
section_style = ParagraphStyle(
    "CareerAISection",
    parent=styles["Heading2"],
    fontName=bold_font,
    fontSize=15,
    leading=20,
    textColor=dark_purple,
    spaceBefore=14,
    spaceAfter=8,
)
body_style = ParagraphStyle(
    "CareerAIBody",
    parent=styles["BodyText"],
    fontName=regular_font,
    fontSize=9.5,
    leading=14,
    textColor=ink,
    spaceAfter=5,
)
small_style = ParagraphStyle(
    "CareerAISmall",
    parent=body_style,
    fontSize=8,
    leading=11,
    textColor=muted,
)
question_style = ParagraphStyle(
    "CareerAIQuestion",
    parent=body_style,
    fontName=bold_font,
    fontSize=11,
    leading=15,
    textColor=ink,
    spaceAfter=6,
)
answer_label_style = ParagraphStyle(
    "CareerAIAnswerLabel",
    parent=body_style,
    fontName=bold_font,
    fontSize=8.5,
    leading=11,
    textColor=purple,
    spaceAfter=4,
)
center_small_style = ParagraphStyle(
    "CareerAICenterSmall",
    parent=small_style,
    alignment=TA_CENTER,
)

buffer = BytesIO()
doc = SimpleDocTemplate(
    buffer,
```

```
        pagesize=A4,
        rightMargin=17 * mm,
        leftMargin=17 * mm,
        topMargin=18 * mm,
        bottomMargin=17 * mm,
        title="CareerAI Interview Report",
        author="CareerAI",
    )

    def draw_header_footer(canvas, document):
        canvas.saveState()
        width, height = A4
        canvas.setFillColor(purple)
        canvas.rect(0, height - 8 * mm, width, 8 * mm, fill=1, stroke=0)
        canvas.setFont("Helvetica-Bold", 8)
        canvas.setFillColor(colors.white)
        canvas.drawString(17 * mm, height - 5.2 * mm, "CareerAI")
        canvas.setFont("Helvetica", 7.5)
        canvas.setFillColor(muted)
        canvas.drawString(17 * mm, 9 * mm, "CareerAI · Resume-grounded interview practice")
        canvas.drawRightString(width - 17 * mm, 9 * mm, f"Page {document.page}")
        canvas.restoreState()

    feedback = getattr(session, "feedback", None)
    feedback = feedback if feedback is not None else type("EmptyFeedback", (), {})()
    questions = list(session.questions.all().prefetch_related("answer"))
    answer_map = _answer_map(feedback)
    story = []

    story.append(Spacer(1, 5 * mm))
    story.append(Paragraph("CareerAI Interview Report", title_style))
    started = getattr(session, "started_at", None)
    date_text = started.strftime("%d %b %Y") if started else ""
    story.append(
        Paragraph(
            _paragraph_text(f"{session.role} · {str(session.difficulty).title()} · {date_text}"),
            subtitle_style,
        )
    )

    score = float(session.overall_score or 0)
    score_table = Table(
        [
            [
                Paragraph("<b>OVERALL SCORE</b><br/><font size=24 color='#312e81'>" + f"{score:.1f}" +
"</font><font size=10 color='#6b7280'> / 100</font>", center_small_style),
                Paragraph(_paragraph_text(f"{len(questions)} questions completed"), center_small_style),
                Paragraph(_paragraph_text(str(getattr(feedback, "generated_by", "local")).upper() + "
FEEDBACK"), center_small_style),
            ]
        ],
        colWidths=[55 * mm, 55 * mm, 55 * mm],
    )
    score_table.setStyle(
        TableStyle(
            [
                ("BACKGROUND", (0, 0), (-1, -1), light_purple),
                ("BOX", (0, 0), (-1, -1), 0.7, colors.HexColor("#ddd6fe")),
                ("INNERGRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#e9e5ff")),
                ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
                ("TOPPADDING", (0, 0), (-1, -1), 12),
                ("BOTTOMPADDING", (0, 0), (-1, -1), 12),
            ]
```

```
        )
    )
    story.append(score_table)
    story.append(Spacer(1, 4 * mm))

    story.append(Paragraph("Coach summary", section_style))
    summary_data = []
    for label, values in (
        ("Strengths", _list_items(getattr(feedback, "strengths", []))),
        ("Focus areas", _list_items(getattr(feedback, "weaknesses", []))),
        ("Next steps", _list_items(getattr(feedback, "recommendations", []))),
    ):
        rendered = "<br/>".join(f"• {_paragraph_text(value)}" for value in values) or "-"
        summary_data.append([Paragraph(f"<b>{html.escape(label)}</b>", body_style), Paragraph(rendered,
body_style)])
    summary_table = Table(summary_data, colWidths=[32 * mm, 133 * mm], hAlign="LEFT")
    summary_table.setStyle(
        TableStyle(
            [
                ("BACKGROUND", (0, 0), (0, -1), light_purple),
                ("BOX", (0, 0), (-1, -1), 0.5, line),
                ("INNERGRID", (0, 0), (-1, -1), 0.35, line),
                ("VALIGN", (0, 0), (-1, -1), "TOP"),
                ("LEFTPADDING", (0, 0), (-1, -1), 8),
                ("RIGHTPADDING", (0, 0), (-1, -1), 8),
                ("TOPPADDING", (0, 0), (-1, -1), 7),
                ("BOTTOMPADDING", (0, 0), (-1, -1), 7),
            ]
        )
    )
    story.append(summary_table)
    story.append(Paragraph("Professional answers are practice models. Replace any generic wording with your
exact contribution and verified results before using it in a real interview.", small_style))
    story.append(PageBreak())
    story.append(Paragraph("Question-by-question answer key", section_style))
    story.append(Paragraph("Every answer is grounded in the resume evidence used to create this interview and
is provided in English and Hindi.", subtitle_style))

    for index, question in enumerate(questions, start=1):
        item = answer_map.get(int(question.order), {})
        professional_en = item.get("answer_en", "Professional answer was not stored for this question.")
        professional_hi = item.get("answer_hi", "इस प्रश्न के लिए professional answer उपलब्ध नहीं है।")
        evidence = item.get("source_evidence", question.source_evidence)
        candidate_answer = getattr(getattr(question, "answer", None), "answer_text", "")
        candidate_score = getattr(getattr(question, "answer", None), "overall_score", None)
        heading = Paragraph(f"<font color='#4f46e5'><b>Q{index:02d}</b></font>
{_paragraph_text(question.question_text)}", question_style)
        evidence_text = Paragraph(f"<b>Resume evidence:</b> {_paragraph_text(evidence)}", small_style)
        english_box = Table([[Paragraph("PROFESSIONAL ANSWER · ENGLISH", answer_label_style),
Paragraph(_paragraph_text(professional_en), body_style)]], colWidths=[45 * mm, 120 * mm])
        english_box.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#f8faff")),
            ("BOX", (0, 0), (-1, -1), 0.5, colors.HexColor("#c7d2fe")),
            ("LEFTPADDING", (0, 0), (-1, -1), 9), ("RIGHTPADDING", (0, 0), (-1, -1), 9),
            ("TOPPADDING", (0, 0), (-1, -1), 7), ("BOTTOMPADDING", (0, 0), (-1, -1), 7),
        ]))
        hindi_box = Table([[Paragraph("PROFESSIONAL ANSWER · हिन्दी", answer_label_style),
Paragraph(_paragraph_text(professional_hi), body_style)]], colWidths=[45 * mm, 120 * mm])
        hindi_box.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#fffdf7")),
            ("BOX", (0, 0), (-1, -1), 0.5, colors.HexColor("#fde68a")),
            ("LEFTPADDING", (0, 0), (-1, -1), 9), ("RIGHTPADDING", (0, 0), (-1, -1), 9),
            ("TOPPADDING", (0, 0), (-1, -1), 7), ("BOTTOMPADDING", (0, 0), (-1, -1), 7),
```

```
        ]))
        parts = [heading, evidence_text, Spacer(1, 1.5 * mm), english_box, Spacer(1, 2 * mm), hindi_box]
        if candidate_answer:
            score_text = f" · Score: {float(candidate_score):.1f}/100" if candidate_score is not None else ""
            candidate_box = Table([[Paragraph(f"<b>YOUR
ANSWER{html.escape(score_text)}</b><br/>{_paragraph_text(candidate_answer)}", small_style)]], colWidths=[165 *
mm])
            candidate_box.setStyle(TableStyle([
                ("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#f9fafb")),
                ("BOX", (0, 0), (-1, -1), 0.5, line),
                ("LEFTPADDING", (0, 0), (-1, -1), 9), ("RIGHTPADDING", (0, 0), (-1, -1), 9),
                ("TOPPADDING", (0, 0), (-1, -1), 7), ("BOTTOMPADDING", (0, 0), (-1, -1), 7),
            ]))
            parts.extend([Spacer(1, 2 * mm), candidate_box])
        parts.append(Spacer(1, 6 * mm))
        story.append(KeepTogether(parts))

    story.append(Spacer(1, 5 * mm))
    story.append(Paragraph("End of report · Keep practising with a new resume-grounded interview to get a
different question set.", small_style))
    doc.build(story, onFirstPage=draw_header_footer, onLaterPages=draw_header_footer)
    return buffer.getvalue()
```

M.8 users tests.py

```
import json
from unittest.mock import patch

from django.contrib.auth.models import User
from django.core.files.uploadedfile import SimpleUploadedFile
from django.test import TestCase, override_settings

from .models import InterviewAnswer, InterviewSession, Resume
from .resume_services import build_local_profile


class ResumeInterviewTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username="candidate",
            password="StrongPass123!",
        )
        self.client.force_login(self.user)

    def test_anonymous_dashboard_redirects_to_login(self):
        self.client.logout()
        response = self.client.get("/dashboard/")
        self.assertRedirects(response, "/login/?next=/dashboard/")

    def test_upload_rejects_non_pdf(self):
        response = self.client.post(
            "/upload/",
            {"resume": SimpleUploadedFile("resume.docx", b"not a pdf")},
        )
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Only PDF resumes are supported")
        self.assertEqual(Resume.objects.count(), 0)

    @patch("users.views.extract_resume_text")
    def test_upload_builds_local_profile(self, extract):
        extract.return_value = (
            "Education\nDiploma Computer Engineering\n"
            "Skills\nPython, Django, MySQL\n"
            "Projects\nAI Interview Simulator - Django project"
        )
        response = self.client.post(
            "/upload/",
            {
                "resume": SimpleUploadedFile(
                    "resume.pdf",
                    b"%PDF-1.4 fake test payload",
                    content_type="application/pdf",
                )
            },
        )
        self.assertEqual(response.status_code, 200)
        resume = Resume.objects.get(user=self.user)
        self.assertEqual(resume.parse_status, "complete")
        self.assertIn("Django", resume.parsed_profile["skills"])
        self.assertContains(response, "Local resume analysis ready")

    def test_offline_interview_saves_answer_and_report(self):
        raw_text = (
            "Education\nDiploma Computer Engineering\n"
            "Skills\nPython, Django, MySQL\n"
            "Projects\nAI Interview Simulator - Django project"
```

```
        )
        resume = Resume.objects.create(
            user=self.user,
            resume_file=SimpleUploadedFile("resume.pdf", b"%PDF-1.4"),
            raw_text=raw_text,
            parsed_profile=build_local_profile(raw_text),
            parse_status="complete",
        )
        response = self.client.post(
            "/api/interview/start/",
            data=json.dumps(
                {
                    "role": "Software Developer",
                    "difficulty": "medium",
                    "interview_type": "general",
                    "count": 5,
                }
            ),
            content_type="application/json",
        )
        self.assertEqual(response.status_code, 200)
        payload = response.json()
        self.assertEqual(payload["generated_by"], "local")
        self.assertGreaterEqual(len(payload["questions"]), 1)

        question_id = payload["questions"][0]["id"]
        answer_response = self.client.post(
            "/api/interview/answer/",
            data=json.dumps(
                {
                    "question_id": question_id,
                    "answer": "I used Django and Python to build the project and implemented authentication.",
                }
            ),
            content_type="application/json",
        )
        self.assertEqual(answer_response.status_code, 200)
        self.assertEqual(InterviewAnswer.objects.count(), 1)

        finish_response = self.client.post(
            "/api/interview/finish/",
            data=json.dumps({"session_id": payload["session_id"]}),
            content_type="application/json",
        )
        self.assertEqual(finish_response.status_code, 200)
        session = InterviewSession.objects.get(pk=payload["session_id"])
        self.assertEqual(session.status, "completed")
        self.assertIsNotNone(session.overall_score)


class PasswordResetTests(TestCase):
    @override_settings(EMAIL_BACKEND="django.core.mail.backends.locmem.EmailBackend")
    @patch("users.views.secrets.randbelow", return_value=123456)
    def test_valid_otp_persists_new_password_and_rejects_old_password(self, _randbelow):
        user = User.objects.create_user(
            username="reset-candidate",
            email="reset-candidate@example.com",
            password="OldStrongPass123!",
        )
        already_logged_in = self.client.__class__()
        already_logged_in.force_login(user)

        response = self.client.post("/forgot-password/", {"email": user.email})
```

```
        self.assertRedirects(response, "/forgot-password/verify/")
        response = self.client.post("/forgot-password/verify/", {"otp": "123456"})
        self.assertRedirects(response, "/forgot-password/new/")
        response = self.client.post(
            "/forgot-password/new/",
            {
                "password": "NewStrongPass123!X",
                "confirm_password": "NewStrongPass123!X",
            },
        )
        self.assertRedirects(response, "/login/")

        user.refresh_from_db()
        self.assertTrue(user.check_password("NewStrongPass123!X"))
        self.assertFalse(user.check_password("OldStrongPass123!"))

        old_password_response = self.client.post(
            "/login/",
            {"username": user.username, "password": "OldStrongPass123!"},
        )
        self.assertEqual(old_password_response.status_code, 200)
        self.assertContains(old_password_response, "Invalid username or password.")

        new_password_response = self.client.post(
            "/login/",
            {"username": user.username, "password": "NewStrongPass123!X"},
        )
        self.assertRedirects(new_password_response, "/dashboard/")

        # A session that was open before recovery is also invalidated.
        stale_session_response = already_logged_in.get("/dashboard/")
        self.assertRedirects(stale_session_response, "/login/?next=/dashboard/")

    def test_performance_is_private_and_dashboard_links_to_it(self):
        other_user = User.objects.create_user(
            username="other-candidate",
            password="OtherStrongPass123!",
        )
        InterviewSession.objects.create(
            user=other_user,
            role="Private role",
            status="completed",
            question_count=15,
            overall_score=99,
        )
        self.client.force_login(
            User.objects.create_user(
                username="performance-candidate",
                password="PerformanceStrongPass123!",
            )
        )
        dashboard = self.client.get("/dashboard/")
        self.assertEqual(dashboard.status_code, 200)
        self.assertContains(dashboard, 'href="/performance/"')
        performance = self.client.get("/performance/")
        self.assertEqual(performance.status_code, 200)
        self.assertContains(performance, "No completed interviews yet")
        self.assertNotContains(performance, "Private role")

    def test_unknown_reset_email_uses_neutral_response(self):
        response = self.client.post(
            "/forgot-password/",
            {"email": "does-not-exist@example.com"},
```

```
        )
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "If an active CareerAI account matches this email")
        self.assertNotContains(response, "No active account was found")
        self.assertNotIn("password_reset_otp_hash", self.client.session)

    def test_registration_requires_an_unambiguous_email(self):
        response = self.client.post(
            "/register/",
            {"username": "missing-email", "password": "StrongPass123!"},
        )
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "This field is required")

    def test_malformed_login_post_does_not_raise_server_error(self):
        response = self.client.post("/login/", {})
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Invalid username or password.")
```